
BML Toolbox

Reference Guide

Version 1.0

April 2026

Brain Modulation Laboratory

Massachusetts General Hospital · University of Pittsburgh

*A comprehensive reference for annotation table manipulation
and multi-device clock synchronization in MATLAB.*

Document	BML Toolbox Reference Guide
Version	1.0
Date	April 2026
Language	MATLAB (Python migration in progress)
Devices supported	Ripple/Trellis, Zoom, NeuroOmega, Psychtoolbox

Version History

Version	Date	Changes
1.0	April 2026	First unified release. Merged annotation algebra guide (Part I) with synchronization pipeline guide (Part II). Added: digital-trigger audio sync, task-event fallback strategy, old vs. new DP matching comparison, Python migration roadmap, best practices, complete worked examples, ReadTheDocs site.
0.2	March 2026	Sync guide: added Psychtoolbox sync, expanded consolidation section, data flow diagram.
0.1	January 2026	Initial sync guide draft covering Zoom analog sync, NeuroOmega event sync, bml_timealign/bml_timewarp internals.

Introduction

About this guide

This guide is the complete reference for the **BML (Brain Modulation Laboratory) MATLAB toolbox**, used for intraoperative and clinical neurophysiology data analysis at MGH and the University of Pittsburgh. It covers:

Part I: Annotation Tables

The universal data structure used throughout BML. Every time-stamped object — a trial, a spike, a file, an artifact window — is an annotation table. Part I covers the schema, all set/interval algebra operations (filter, intersect, union, difference, consolidate, ...), I/O, visualization, and a complete worked example.

Part II: Synchronization

How to align recordings from four independent devices (Ripple/Trellis, Zoom audio recorder, NeuroOmega, task laptop) onto a single master clock. Covers analog cross-correlation, digital event matching, the DP algorithm, the fallback strategy, consolidation, and best practices.

Design philosophy

Three principles underlie every BML function:

1. **Annotation tables are the universal contract.** Every function takes annotation tables as input and returns annotation tables as output. This makes functions composable: the output of `bml_annot_filter` feeds directly into `bml_annot_intersect` without any data transformation.
2. **All times are on the master clock.** After synchronization, every sample index and every event timestamp is expressed in seconds on the Trellis/Ripple hardware clock. You never need to know which device produced a data point.
3. **Non-destructive.** Raw files are never modified. Sync state is stored separately in `sync_roi` tables. Any step can be re-run without corrupting the original data.

How to use this guide

- **New to BML?** Start with Section [0.1](#) (annotation table schema), then Section [0.2](#) (the ROI table and coordinate system), then the worked example in Chapter [0.20](#).
- **Doing sync?** Jump to Part II. The sync overview diagram (Section [.3.2](#)) shows the full pipeline at a glance.
- **Looking for a specific function?** Use the reference table in Chapter [0.21](#) or the table of contents.

Contents

Version History	1
Introduction	2
I Annotation Tables and Interval Algebra	6
0.1 The Annotation Table: Core Data Structure	7
0.1.1 Rationale	7
0.1.2 The four mandatory columns	7
0.1.3 Creating annotation tables	7
0.2 The ROI Table and the (s1, t1, s2, t2) Coordinate System	8
0.2.1 What is an ROI table?	8
0.2.2 The linear coordinate map	8
0.2.3 Why sample midpoints?	8
0.3 Creating and Loading Annotation Tables	9
0.3.1 From scratch	9
0.3.2 From BIDS TSV files (bml_annot_read_tsv)	9
0.3.3 From FieldTrip event structs (bml_event2annot)	10
0.3.4 Scanning a folder to build the initial ROI table	10
0.4 bml_annot_extend — Temporal Dilation and Erosion	10
0.4.1 Rationale	10
0.5 bml_annot_filter / filterout — Selection by Time Window	11
0.5.1 Rationale	11
0.5.2 Overlap condition	11
0.6 bml_annot_intersect — Temporal Intersection	12
0.6.1 Rationale	12
0.7 bml_annot_union — Temporal Union	13
0.7.1 Rationale	13
0.8 bml_annot_difference — Set Subtraction	13
0.8.1 Rationale	13
0.9 bml_annot Consolidate — Merging by Criterion	14
0.9.1 Rationale	14
0.9.2 Algorithm	14
0.10 bml_annot_blocks — Run-Length Encoding and Transitions	15
0.10.1 Rationale	15
0.11 bml_annot_shadow — Filling the Gaps	15
0.11.1 Rationale	15
0.12 bml_annot_coverage — Measuring How Much is Covered	16
0.13 bml_annot_overlap — Detecting Conflicts	17
0.14 Joining Tables	17
0.14.1 bml_annot_left_join — Key-Based Join	17
0.14.2 bml_annot_transfer — Temporal Overlap Join	17
0.15 bml_annot_calculate — Feature Extraction over Epochs	18

0.16	<code>bml_annot_detect</code> — Threshold-Based Event Detection	18
0.17	<code>bml_annot2raw</code> / <code>bml_raw2annot</code> — Signal Encoding	19
0.17.1	Annotations to binary/count signals (<code>bml_annot2raw</code>)	19
0.18	<code>bml_annot_plot</code> — Visualization	19
0.19	<code>bml_annot_to</code> and <code>bml_annot_rowbind</code> — Utilities	20
0.19.1	<code>bml_annot_to</code> — Change Time Reference	20
0.19.2	<code>bml_annot_rowbind</code> — Vertical Concatenation	20
0.20	Complete Worked Example: From Raw Events to Clean Trial Features	20
0.21	Annotation Algebra: Complete Reference Table	22
.1	Interval Algebra: Mathematical Summary	24
II	The Synchronization Pipeline	25
.2	Glossary of Key Terms	1
.3	System Overview and Data Flow	4
.3.1	The synchronization problem	4
.3.2	Complete data flow diagram	4
.4	Annotation Tables and the ROI Structure	5
.4.1	Annotation table schema	5
.4.2	ROI table: the sync state carrier	6
.4.3	Key annotation manipulation functions	6
.5	The Coordinate System	7
.5.1	Why two points?	7
.6	Building the Initial ROI Table	7
.7	Synchronizing Audio (Zoom) to Ripple/Trellis	8
.7.1	Approach A: Shared analog channel (<code>bml_sync_analog</code>)	8
.7.2	Approach B: Digital triggers via audio-peak detection (<code>bml_sync_audio_event</code>)	9
.8	Synchronizing NeuroOmega to Ripple/Trellis	9
.9	Synchronizing Task Laptop (Psychtoolbox) to Ripple	10
.9.1	The setup	10
.9.2	Step 1: Read the task log file	10
.9.3	Step 2: Get the same events from Ripple	11
.9.4	Step 3: Synchronize with <code>bml_sync_digital</code>	11
.9.5	Step 4: Apply sync to task events	11
.9.6	Step 4b: Exact-match fallback strategy	12
.10	Event Alignment: From Brute-Force to Dynamic Programming	13
.10.1	The problem with simple event matching	13
.10.2	The old approach (archived, pre-2025)	14
.10.3	The new approach (current)	14
.10.4	The dynamic programming algorithm (both versions)	15
.10.5	How <code>bml_sync_digital</code> uses the DP output	16
.11	Alignment Function Hierarchy	16
.11.1	Call graph	17
.11.2	The two dimensions	17
.11.3	Decision guide: which function to call?	17
.12	Event Alignment Internals: <code>bml_timealign_annot</code>	18
.13	Continuous Signal Alignment	18
.13.1	<code>bml_timealign</code> — cross-correlation	18
.13.2	<code>bml_timewarp</code> — linear warp optimization	19
.14	Consolidation: What It Is and Why It Matters	19

.14.1	The problem consolidation solves	19
.14.2	Level 1: per-file consolidation	19
.14.3	Level 2: contiguous file consolidation	20
.14.4	What the consolidated table looks like	20
.15	Complete End-to-End Example	20
.16	Common Pitfalls	22
.17	Diagnostic Plots: How to Enable and Interpret	22
.17.1	bml_sync_match_events — DP similarity plot	23
.17.2	bml_sync_audio_event / bml_sync_neuroomega_event — event alignment plot	23
.17.3	bml_sync_digital — residual + alignment plot	24
.17.4	bml_sync_consolidate — pairwise residual heatmap	25
.17.5	Quick-reference: what to check first	26
.18	Best Practices and Lab Conventions	27
.18.1	Choose your sync method by signal quality	27
.18.2	Always validate sync quality	27
.18.3	Annotation tables as the lingua franca	27
.18.4	Save intermediate outputs	27
.18.5	The fallback-first principle	28
.18.6	BIDS event file conventions	28
.18.7	Chunking strategy	28
.18.8	Toward Python: design principles to carry forward	28
.1	Mathematical Reference	29

Part I

Annotation Tables and Interval Algebra

0.1 The Annotation Table: Core Data Structure

0.1.1 Rationale

Every piece of timing information in BML — a stimulus onset, a file on disk, an artifact window, a sync interval, a spike — is stored in a single, uniform structure: the **annotation table**. This makes every function interoperable: a function that accepts an annotation table accepts *any* kind of time-stamped data.

The key insight is that everything interesting in neuroscience has a **start time** and an **end time**. Even instantaneous events (a spike, a button press) are stored with `ends = starts`, so the same code handles both.

0.1.2 The four mandatory columns

id	starts	ends	duration	extra cols...
1	10.000	10.500	0.500	'STIM_ON'
2	11.200	11.700	0.500	'STIM_ON'
3	13.500	14.200	0.700	'RESPONSE'
↓ auto-assigned integer	↓ seconds on master clock	↓ ends—starts auto-computed	↓ ends—starts auto-computed	↓ arbitrary extra data

- **id**: unique integer per row. Auto-assigned in ascending order sorted by starts. Not stable across calls to `bml_annot_table` — never use `id` as a persistent key.
- **starts**: interval start time, in seconds on the master clock.
- **ends**: interval end time ($\geq$ starts). Instantaneous events: `ends = starts`.
- **duration**: `ends - starts`, always recomputed; never trust an existing duration column if starts/ends were modified.

0.1.3 Creating annotation tables

```

1 % 1. From a table with starts and ends columns
2 T = table([1.0; 2.5; 4.0], [1.5; 3.0; 4.8], ...
3           'VariableNames', {'starts','ends'});
4 T.type = {'STIM_ON'; 'STIM_ON'; 'RESPONSE'};
5 annot = bml_annot_table(T);
6 % -> id=[1;2;3], duration=[0.5;0.5;0.8] auto-computed
7
8 % 2. From a two-column numeric matrix [starts, ends]
9 annot = bml_annot_table([10 10.5; 11.2 11.7; 13.5 14.2]);
10
11 % 3. From a struct
12 s.starts = [10; 11.2; 13.5];
13 s.ends   = [10.5; 11.7; 14.2];
14 s.type   = {'A'; 'A'; 'B'};
15 annot = bml_annot_table(struct2table(s));

```

Listing 1: Three ways to call `bml_annot_table`

Warning

`bml_annot_table()` sorts rows by `starts` and re-numbers `id` from 1. If you add a row manually and call it again, all `ids` change. Use `id` only as a within-table reference, never as a cross-table key.

0.2 The ROI Table and the (s1, t1, s2, t2) Coordinate System

0.2.1 What is an ROI table?

A **Region of Interest (ROI) table** is an annotation table extended with file metadata and *sync coordinates*. Each row describes a time window within a specific file on disk.

Column	Type	Source	Meaning
id, starts, ends, duration	double	annotation	Time window (master clock)
folder	string	filesystem	Directory containing the file
name	string	filesystem	Filename
nSamples	int	file header	Total number of samples
Fs	double	file header	Nominal sampling rate (Hz)
filetype	string	config	e.g. 'trellis', 'zoom', 'neuroomega'
chantype	string	config	e.g. 'analog', 'digital', 'lfp'
s1	int	sync	First anchor: sample index
t1	double	sync	First anchor: master-clock time of sample s1
s2	int	sync	Second anchor: sample index
t2	double	sync	Second anchor: master-clock time of sample s2

0.2.2 The linear coordinate map

The four sync coordinates (`s1`, `t1`, `s2`, `t2`) define an affine map between sample index space and master-clock time:

Index-to-time conversion

$$F_{s,\text{eff}} = \frac{s_2 - s_1}{t_2 - t_1}, \quad t(i) = \frac{i - s_1}{F_{s,\text{eff}}} + t_1 - \frac{0.5}{F_{s,\text{eff}}} = \frac{i}{F_{s,\text{eff}}} - \frac{0.5}{F_{s,\text{eff}}} + \frac{s_2 t_1 - t_2 s_1}{s_2 - s_1}$$

Time-to-index conversion

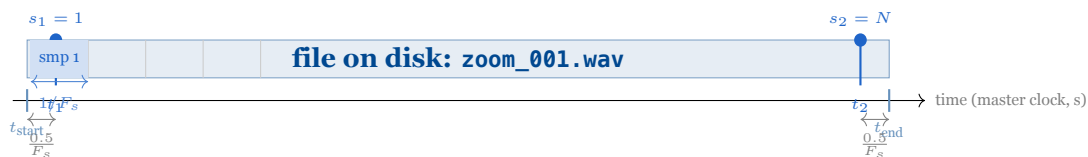
$$i(t) = \text{round}\left(\frac{t_2 s_1 - s_2 t_1 + (s_2 - s_1)t}{t_2 - t_1}\right)$$

0.2.3 Why sample midpoints?

Sample i occupies the time window $\left[\frac{i-1}{F_s}, \frac{i}{F_s}\right)$. Its *midpoint* is $\frac{i}{F_s} - \frac{0.5}{F_s}$. The coordinates `t1` and `t2` refer to the midpoints of samples `s1` and `s2`, respectively.

This matters because: if you know the start of a 30 kHz file is t_{start} , then sample 1 starts at t_{start} but its midpoint (what `t1` records) is $t_{\text{start}} + \frac{0.5}{30000}$. The default initialization is:

$$s_1 = 1, \quad t_1 = \text{starts} + \frac{0.5}{F_s}, \quad s_2 = N_{\text{samples}}, \quad t_2 = \text{ends} - \frac{0.5}{F_s}$$



```

1 % Build from file scan
2 roi = bml_roi_table(bml_info_raw(struct('folder','/data/p001/s01')));
3
4 % Default coordinates:
5 % roi.s1 = 1 (first sample)
6 % roi.t1 = roi.starts + 0.5/roi.Fs (midpoint of sample 1)
7 % roi.s2 = roi.nSamples
8 % roi.t2 = roi.ends - 0.5/roi.Fs (midpoint of last sample)
9
10 % After synchronization, t1/t2 are updated to master-clock times.
11 % Convert between sample index and time:
12 t = bml_idx2time(roi(1,:), 15000); % time of sample 15000
13 i = bml_time2idx(roi(1,:), 52312.5); % sample nearest to 52312.5 s

```

Listing 2: Building and using an ROI table

0.3 Creating and Loading Annotation Tables

0.3.1 From scratch

```

1 % Minimal: just starts and ends
2 spikes = bml_annot_table(table([1.2;1.8;2.4],[1.2;1.8;2.4],...
3 'VariableNames',{'starts','ends'}));
4
5 % With extra columns
6 trials = table([10;20;30], [10.5;20.5;30.5], {'go';'nogo';'go'}, [1;2;3],...
7 'VariableNames',{'starts','ends','trial_type','value'});
8 trials = bml_annot_table(trials);

```

0.3.2 From BIDS TSV files (bml_annot_read_tsv)

BIDS event files use onset (seconds from task start) and duration columns. `bml_annot_read_tsv` renames onset→starts and computes ends = starts + duration.

```

1 events = bml_annot_read_tsv( ...
2 'sub-DM1001_ses-intraop_task-speech_events.tsv');
3 % Columns: id, starts, ends, duration, trial_type, response_time, ...
4 % Also extracts sub/ses/task from filename if AppendColsFromFilename=true
5
6 % Write back after sync:
7 bml_annot_write_tsv(events_synced, 'sub-DM1001_...-sync.tsv');
8 % Renames starts→onset, drops id, writes tab-delimited

```

0.3.3 From FieldTrip event structs (bml_event2annot)

```

1 % Read Ripple (Trellis) events
2 ft_events = ft_read_event(fullfile(roi.folder{1},roi.name{1}));
3
4 cfg = [];
5 cfg.roi = roi(strcmp(roi.filetype,'trellis'),:);
6 cfg.Fs = 30000;
7 annot_events = bml_event2annot(cfg, ft_events);
8 % Columns: id, starts, ends, duration, type, value, sample
9 % sample = raw sample index; starts/ends = master-clock seconds via roi
10
11 % Quick shortcut used throughout BML:
12 master_events = bml_event2annot([], bml_read_event(trellis_roi));

```

0.3.4 Scanning a folder to build the initial ROI table

```

1 % Step 1: scan folder, infer file type from extension/name pattern
2 info = bml_info_raw(struct('folder', '/data/p001/s01', ...
3     'filetype', {'trellis','zoom','neuroomega'}));
4
5 % Step 2: create ROI table with default sync coordinates
6 roi = bml_roi_table(info);
7
8 % roi now has one row per file, with:
9 %   s1=1, t1=starts+0.5/Fs (OS timestamps, unreliable)
10 %   s2=nSamples, t2=ends-0.5/Fs
11 % Sync functions update t1/t2 to master-clock times.

```

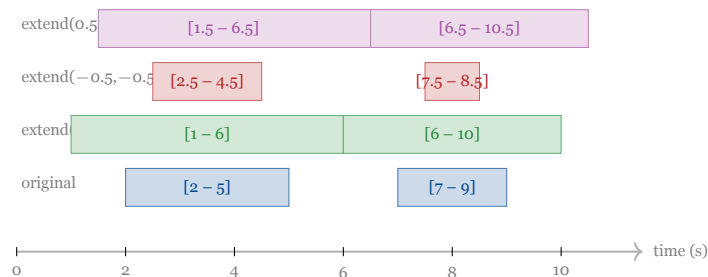
0.4 bml_annot_extend — Temporal Dilation and Erosion

0.4.1 Rationale

Many analysis steps need a *window around* an event rather than the event itself. `bml_annot_extend` adds (or subtracts) time on either side of every row, without touching other columns.

Extension rule

$\text{starts}_{\text{new}} = \text{starts} - \text{ext1}$, $\text{ends}_{\text{new}} = \text{ends} + \text{ext2}$
 Positive = expand outward. Negative = shrink inward (erosion).



```

1 % Add 0.5 s context on each side of every trial
2 trial_windows = bml_annot_extend(trials, 0.5);
3
4 % Pre-stimulus baseline: 0.5 s ending at stimulus onset

```

```

5 baseline = bml_annot_extend(stim_events, 0.5, 0);
6 % Now baseline.ends == stim_events.starts, baseline.starts == stim_events.starts -
  0.5
7
8 % Shrink (erode) artifact windows by 50 ms to be conservative
9 artifacts_trimmed = bml_annot_extend(artifacts, -0.05);
10
11 % Asymmetric: 1 s before onset, 2 s after
12 analysis_win = bml_annot_extend(stim_events, 1.0, 2.0);
13
14 % Sync coarsetol: expand the ROI window to capture master events
15 % that might fall ±100 s outside the file OS timestamps
16 roi_expanded = bml_annot_extend(roi_file, 100);

```

0.5 bml_annot_filter / filterout — Selection by Time Window

0.5.1 Rationale

Often you have a table of events and a table of windows, and you want to keep only the events that fall inside the windows (or remove those that overlap bad periods). This is the most common single operation in a BML pipeline.

0.5.2 Overlap condition

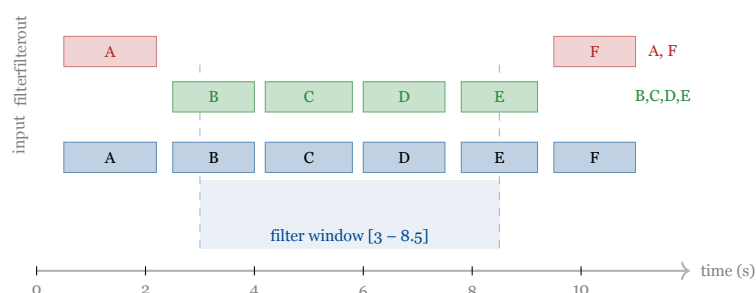
Two intervals $[a_1, a_2]$ and $[b_1, b_2]$ overlap if and only if:

Overlap condition

$$a_1 < b_2 \text{ AND } a_2 > b_1$$

(They overlap when neither is entirely before the other)

With `cfg.overlap = f`: additionally require $|a \cap b|/|a| \geq f$.



Key point: `bml_annot_filter` keeps the rows of `annot` intact (start/end times are not clipped). It is a *selection* operation, not a clip. Use `bml_annot_intersect` if you also want to clip to the filter boundary.

```

1 % Keep only trials within the session window
2 session = bml_annot_table(table(52300, 52600, 'VariableNames',{'starts','ends'}));
3 trials_in_session = bml_annot_filter(trials, session);
4
5 % Remove trials that overlap any artifact
6 clean_trials = bml_annot_filterout(trials, artifacts);

```

```

7
8 % Keep only trials where >= 50% of the trial is inside the analysis window
9 cfg = []; cfg.overlap = 0.5;
10 mostly_inside = bml_annot_filter(cfg, trials, analysis_window);
11
12 % Restrict master events to ±100s of a specific file (sync coarsetol pattern)
13 file_window = bml_annot_extend(roi(3,:), 100); % expand file ROI by 100s
14 local_events = bml_annot_filter(master_events, file_window);

```

0.6 bml_annot_intersect — Temporal Intersection

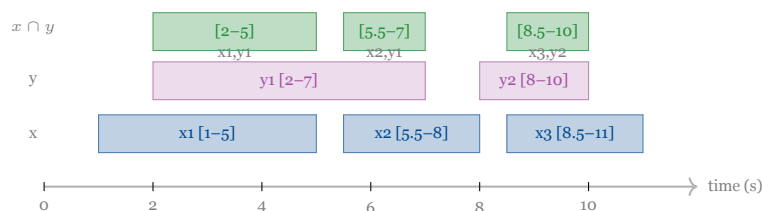
0.6.1 Rationale

`bml_annot_filter` selects whole rows; `bml_annot_intersect` *clips* them to the overlap region. The result intervals are shorter than either input. Crucially, it records where each output interval came from (`x_id`, `y_id`).

Intersection of two intervals

$$[a_1, a_2] \cap [b_1, b_2] = [\max(a_1, b_1), \min(a_2, b_2)]$$

Valid (non-empty) only when $\max(a_1, b_1) < \min(a_2, b_2)$.



Note: *x*₂ overlaps *y*₁ only partially ([5.5-7], not [5.5-8]). *x*₁ and *x*₃ have no intersection with *y*₂; *x*₃ overlaps *y*₂ at [8.5-10], not [8.5-11].

```

1 % Basic intersection: clips trials to session windows
2 clipped = bml_annot_intersect(trials, session_windows);
3 % clipped has x_id (trial id) and y_id (session id) columns
4
5 % Keep columns from both tables (cfg.keep = 'both', default)
6 cfg = []; cfg.keep = 'both';
7 overlap_info = bml_annot_intersect(cfg, trials, artifacts);
8 % overlap_info.trials_id, overlap_info.artifacts_id, plus all extra columns
9
10 % Keep only x columns (strip y metadata from output)
11 cfg = []; cfg.keep = 'x';
12 clipped_trials = bml_annot_intersect(cfg, trials, session_windows);
13
14 % Per-channel intersection (each channel has its own artifact list)
15 cfg = []; cfg.groupby = 'channel';
16 per_ch = bml_annot_intersect(cfg, spikes, artifacts_per_channel);

```

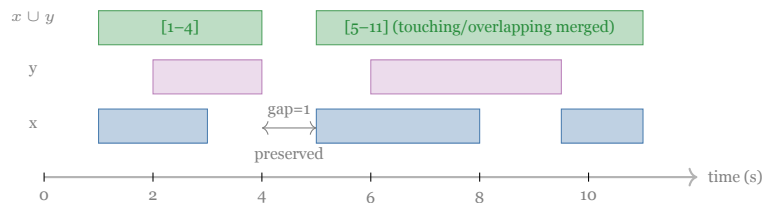
0.7 bml_annot_union — Temporal Union

0.7.1 Rationale

Merge overlapping or touching intervals into the fewest possible non-overlapping intervals. Used to combine artifact lists from multiple sources, merge session windows, etc.

Merging condition

Intervals $[a_1, a_2]$ and $[b_1, b_2]$ (with $b_1 \geq a_1$) are merged when:
 $b_1 \leq a_2 + \text{timetol}$ (they touch or overlap within tolerance)



```

1 % Merge artifact lists from two annotators
2 all_artifacts = bml_annot_union(artifacts_A, artifacts_B);
3
4 % Self-union: consolidate a single table's overlaps
5 % (useful if you added rows that now overlap)
6 clean = bml_annot_union(dirty_annot);
7
8 % Wider tolerance: treat gaps < 10 ms as touching
9 cfg = []; cfg.timetol = 0.01;
10 merged = bml_annot_union(cfg, artifacts_A, artifacts_B);

```

0.8 bml_annot_difference — Set Subtraction

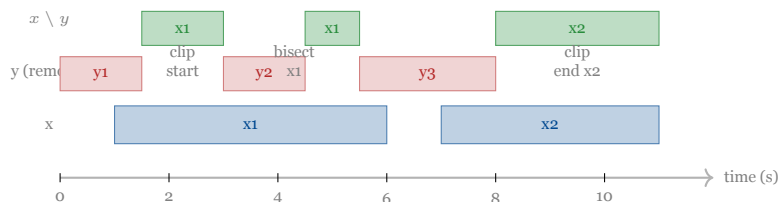
0.8.1 Rationale

Remove regions covered by y from x . The result may split x intervals into pieces. Original x column data is preserved on each fragment.

Set difference

$x \setminus y$: all parts of x that do not overlap y .

If y bisects an x interval: the result has two fragments from the original row.



Five geometric cases handled internally:

1. y fully covers x $\square$ x row removed entirely
2. y clips the start of x $\square$ fragment $[y.\text{ends}, x.\text{ends}]$

3. y clips the end of x $\square$ fragment $[x.starts, y.starts]$
4. y bisects x $\square$ two fragments: $[x.starts, y.starts]$ and $[y.ends, x.ends]$
5. y is outside x $\square$ x row kept as-is

```

1 % Remove artifact periods from trial windows
2 clean_trials = bml_annot_difference(trial_windows, artifacts);
3 % Trials are split where artifacts bisect them.
4 % clean_trials.x_id_ links each fragment back to its original trial.
5
6 % Per-channel artifact removal (artifacts labelled by channel)
7 cfg = []; cfg.groupby_x = 'channel'; cfg.groupby_y = 'channel';
8 clean_per_ch = bml_annot_difference(cfg, lfp_windows, chan_artifacts);
9
10 % What remains after removing all high-gamma bursts from the session?
11 session_gaps = bml_annot_difference(session_window, hg_bursts);

```

0.9 bml_annot_consolidate — Merging by Criterion

0.9.1 Rationale

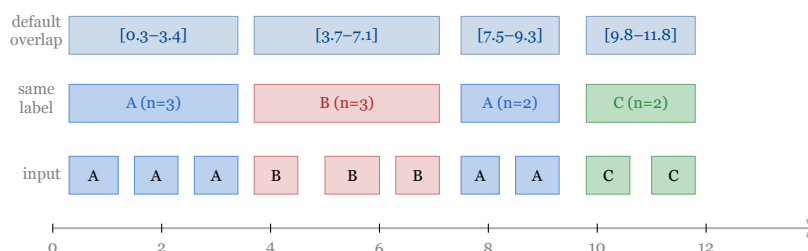
Given a sorted annotation table, consolidate (merge) consecutive rows that satisfy a custom criterion. This is the most flexible grouping primitive in BML: it handles overlap merging, same-label grouping, fixed-size batching, and custom condition merging.

0.9.2 Algorithm

Starting from row 1, greedily try to add each next row to the current group S :

1. Tentatively add row r to S
2. Evaluate $\text{criterion}(S)$
3. If true: keep r in S , continue to next row
4. If false: close current group, start new group with r

The default criterion is overlap/contiguity: $\text{criterion}(S) = S.starts(end) \leq \max(S.ends(1:end-1))$.



```

1 % 1. Default criterion: merge overlapping/touching (same as bml_annot_union)
2 cons = bml_annot_consolidate(annot);
3
4 % 2. Same-label grouping (like run-length encoding)
5 cfg = []; cfg.criterion = @(x) length(unique(x.label)) == 1;
6 cons_by_label = bml_annot_consolidate(cfg, annot);
7
8 % 3. Fixed group size (batch trials in groups of 4)
9 cfg = []; cfg.criterion = @(x) height(x) <= 4;
10 batches = bml_annot_consolidate(cfg, trials);
11

```

```

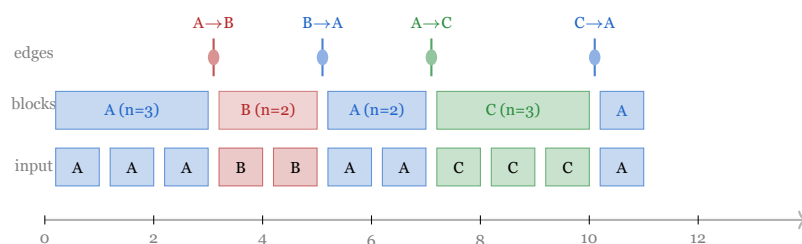
12 % 4. Contiguous files with same depth (NeuroOmega sync pattern)
13 cfg = [];
14 cfg.criterion = @(x) (length(unique(x.depth))==1) && ...
15     (abs((max(x.ends)-min(x.starts)) - sum(x.duration)) < 1e-2);
16 cons_depth = bml_annot_consolidate(cfg, neuro_info);
17
18 % Output columns always include:
19 %   cons_duration = sum(duration of merged rows)
20 %   id_starts     = id of first row in group
21 %   id_ends       = id of last row in group
22 %   cons_n        = number of rows merged

```

0.10 bml_annot_blocks — Run-Length Encoding and Transitions

0.10.1 Rationale

Given a sequence of annotations where each row has a label (e.g., a stimulus category, a behavioral state, a coding label), `bml_annot_blocks` groups consecutive rows with the *same label* into blocks and records the *transitions* between blocks. This is run-length encoding applied to time-series annotations.



```

1 % Detect blocks of same stimulus condition
2 cfg = []; cfg.label = 'trial_type';
3 [blocks, edges] = bml_annot_blocks(cfg, trial_annot);
4 % blocks: id, starts, ends, duration, label, cons_n (how many rows merged)
5 % edges: id, starts, ends, pre_label, post_label
6
7 % With win_edges: create a window around each transition
8 cfg = []; cfg.label = 'depth'; cfg.win_edges = 2.0; % ±2s window
9 [blocks, edges] = bml_annot_blocks(cfg, recording_info);
10 % edges.starts = transition_time - 2.0
11 % edges.ends   = transition_time + 2.0
12
13 % Group by an additional variable (e.g. per session)
14 cfg = []; cfg.label = 'state'; cfg.group_by = 'session_id';
15 [blocks, edges] = bml_annot_blocks(cfg, behavioral_coding);

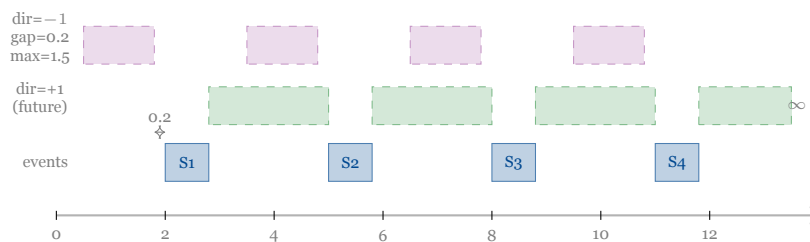
```

0.11 bml_annot_shadow — Filling the Gaps

0.11.1 Rationale

Given a set of events, generate annotations that occupy the *space between* them. The primary use case is creating pre-stimulus baselines: for each stimulus, get the silence that

precedes it.



```

1 % Pre-stimulus baseline: window before each stimulus (gap 0, max 1.5s)
2 cfg = [];
3 cfg.direction = -1; % look into the past
4 cfg.gap_duration = 0; % shadow touches the event
5 cfg.max_duration = 1.5; % cap at 1.5 s
6 baseline = bml_annot_shadow(cfg, stim_events);
7 % baseline(i).ends == stim_events(i).starts (gap=0)
8 % baseline(i).starts == stim_events(i).starts - min(1.5, gap_to_prev_event)
9
10 % Post-response silence (future direction, with 200 ms gap)
11 cfg = []; cfg.direction = 1; cfg.gap_duration = 0.2; cfg.max_duration = 2.0;
12 post_resp = bml_annot_shadow(cfg, response_events);
13
14 % The last shadow always extends to ±inf (clipped by max_duration if set).
15 % Use bml_annot_filter afterward to restrict to a session window:
16 post_resp = bml_annot_filter(post_resp, session_window);

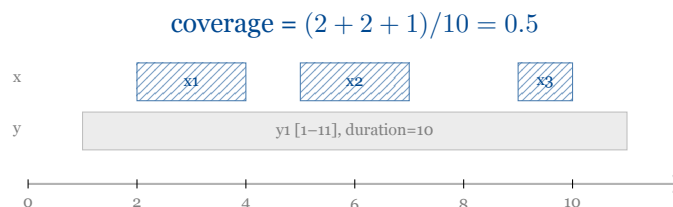
```

0.12 bml_annot_coverage — Measuring How Much is Covered

Coverage formula

$$\text{coverage}(y_j) = \frac{\sum_i |x_i \cap y_j|}{|y_j|}$$

Returns a value in $[0, 1]$: fraction of y_j 's duration covered by x .



```

1 % Fraction of each trial covered by artifacts
2 cov = bml_annot_coverage(artifacts, trials);
3 % cov has same columns as trials plus 'coverage' (0 to 1)
4
5 % Keep only trials with < 10% artifact coverage
6 clean_trials = trials(cov.coverage < 0.1, :);
7
8 % Per-channel: artifact coverage per trial per channel
9 cfg = []; cfg.groupby_x = 'channel';

```

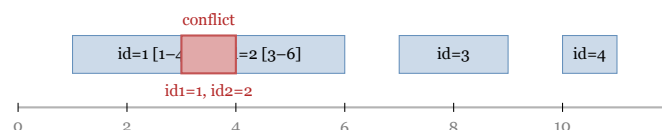
```

10 cov_per_ch = bml_annot_coverage(cfg, chan_artifacts, trials);
11 % One row per (trial x channel) combination
12
13 % Use custom column name
14 cfg = []; cfg.colname = 'artifact_fraction';
15 cov = bml_annot_coverage(cfg, artifacts, trials);

```

0.13 bml_annot_overlap — Detecting Conflicts

`bml_annot_overlap` finds all pairs of rows within the *same* table that overlap each other. The result lists the overlapping pair IDs and the overlap region. Used for validation and debugging.



```

1 % Validate that a table has no overlaps (required before passing as y to intersect)
2 ovlp = bml_annot_overlap(annot);
3 assert isempty(ovlp), 'Table has %d overlapping pairs', height(ovlp);
4
5 % Find and inspect overlapping pairs
6 ovlp = bml_annot_overlap(my_events);
7 if ~isempty(ovlp)
8     disp(ovlp); % id1, id2, starts, ends of each overlap
9     % Fix: resolve by merging or trimming conflicting rows
10    my_events = bml_annot_union(my_events); % merge overlapping pairs
11 end

```

0.14 Joining Tables

0.14.1 bml_annot_left_join — Key-Based Join

Standard relational left join: all rows of the left table are preserved; columns from the right table are added where the key matches.

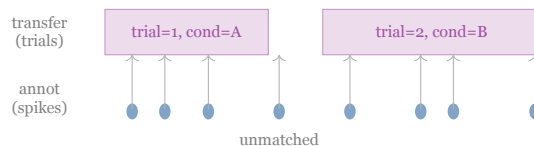
```

1 % Add subject/session metadata to each trial
2 cfg = []; cfg.keys = {'session_id'};
3 trials_with_meta = bml_annot_left_join(cfg, trials, session_metadata);
4
5 % Add per-electrode impedance to spike annotations
6 cfg = []; cfg.keys = {'channel'};
7 spikes_full = bml_annot_left_join(cfg, spikes, electrode_info);

```

0.14.2 bml_annot_transfer — Temporal Overlap Join

For each row in `annot`, find which row(s) in `transfer` contain it (by time overlap), and copy selected columns. Useful for assigning epoch-level labels to individual events within the epoch.



```

1 % Assign trial condition label to each spike
2 cfg = []; cfg.select = {'trial_id', 'condition', 'response'};
3 spikes_labeled = bml_annot_transfer(cfg, spikes, trial_epochs);
4 % Each spike gets the trial_id/condition/response of the trial containing it.
5 % Spikes not inside any trial get NaN/empty for transferred columns.
6
7 % cfg.overlap: require spike to be at least 50% inside the epoch
8 cfg.overlap = 0.5;

```

0.15 bml_annot_calculate — Feature Extraction over Epochs

Applies scalar functions to raw signal data over each epoch defined by an annotation table. Each function receives a FT_DATATYPE_RAW segment and must return a scalar (or one scalar per channel).

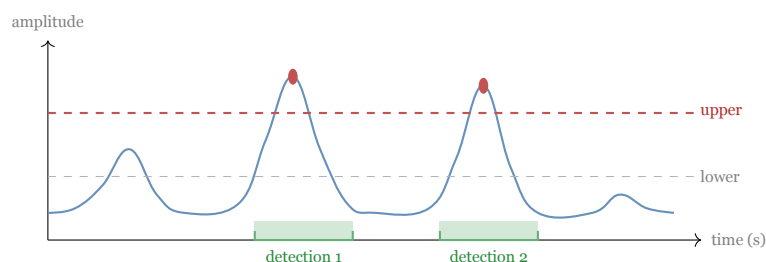
```

1 % Define feature functions
2 rms_fn = @(raw) sqrt(mean(raw.trial{1}(1,:).^2));
3 peak2pk = @(raw) max(raw.trial{1}(1,:)) - min(raw.trial{1}(1,:));
4 log_pwr = @(raw) 10*log10(mean(raw.trial{1}(1,:).^2));
5
6 % Compute features for each trial
7 cfg = [];
8 cfg.roi = sync_roi(strcmp(sync_roi.filetype, 'trellis'),:);
9 cfg.epoch = trial_windows;
10 trial_features = bml_annot_calculate(cfg, trial_windows, ...
11 'rms', rms_fn, 'peak2peak', peak2pk, 'log_power', log_pwr);
12 % trial_features has all original columns plus rms, peak2peak, log_power
13
14 % bml_annot_describe: summary statistics per group
15 cfg = []; cfg.groupby = 'condition';
16 stats = bml_annot_describe(cfg, trial_features);
17 % stats: condition, variable, mean, median, std, min, max, count

```

0.16 bml_annot_detect — Threshold-Based Event Detection

Detects epochs where a signal exceeds a threshold. The algorithm finds peaks above the upper threshold, then expands each detection to the nearest zero-crossing of the lower threshold.



```

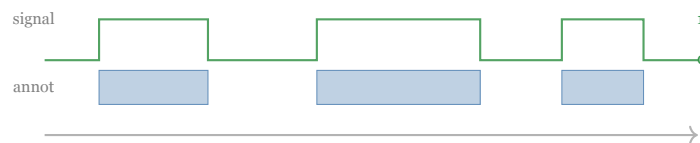
1 % Detect high-gamma (HG) bursts above 3 std
2 hg_raw = bml_load_continuous(struct('roi', hg_roi));
3
4 cfg = [];
5 cfg.threshold = [1.0, 3.0]; % [lower, upper] in units of the signal
6 cfg.channel = 'G14';
7 cfg.max_annots = 500;
8 hg_bursts = bml_annot_detect(cfg, hg_raw);
9 % hg_bursts: starts, ends, env_max, trial, label
10
11 % Single threshold (upper only; lower = 0 crossing)
12 cfg = []; cfg.threshold = 3.0;
13 detections = bml_annot_detect(cfg, envelope_raw);

```

0.17 bml_annot2raw / bml_raw2annot — Signal Encoding

0.17.1 Annotations to binary/count signals (bml_annot2raw)

Converts an annotation table to a continuous FT_DATATYPE_RAW signal: 1 where a row covers that sample, 0 elsewhere. With `cfg.count=true`, counts overlapping annotations per sample.



```

1 % Create binary stimulus presence signal (for cross-correlation with neural data)
2 cfg = [];
3 cfg.roi = trellis_roi;
4 cfg.label = 'stim_channel';
5 stim_raw = bml_annot2raw(cfg, stim_annotations);
6
7 % Count overlapping artifacts per sample (for quality metric)
8 cfg = []; cfg.count = true;
9 artifact_count_raw = bml_annot2raw(cfg, artifacts, trellis_roi);
10 % Each sample = number of simultaneously overlapping artifact annotations
11
12 % Per-label channels: one channel per unique label value
13 cfg = []; cfg.label_colname = 'trial_type';
14 coding_raw = bml_annot2raw(cfg, trial_annotations, roi);
15 % coding_raw has one channel per trial_type
16
17 % Inverse: extract time bounds from a raw struct back to annotation metadata
18 annot_meta = bml_raw2annot([], lfp_raw);
19 % annot_meta: starts, ends, trial, s1, t1, s2, t2, Fs, nSamples

```

0.18 bml_annot_plot — Visualization

Plots annotation intervals as horizontal line segments, with optional faceting (subplots) and y-axis grouping.

```

1 % Basic: one line per annotation id
2 bml_annot_plot([], trial_windows);
3
4 % Group by trial_type on y-axis
5 cfg = []; cfg.y = 'trial_type';
6 bml_annot_plot(cfg, trial_windows);
7
8 % Facet by session (one subplot per session)
9 cfg = [];
10 cfg.y = 'channel'; % y-axis = channel number
11 cfg.facet = 'session_id'; % one panel per session
12 bml_annot_plot(cfg, lfp_epochs);
13
14 % Use in a pipeline to inspect intermediate results
15 bml_annot_plot([], bml_annot_filter(events, session));

```

0.19 bml_annot_to and bml_annot_rowbind — Utilities

0.19.1 bml_annot_to — Change Time Reference

Shifts all times by a constant. Used when you need times relative to an event (e.g., stimulus onset = 0).

Time shift

$$\text{starts}_{\text{new}} = \text{starts} - t_0, \quad \text{ends}_{\text{new}} = \text{ends} - t_0$$

```

1 % Set stimulus onset as t=0
2 cfg = []; cfg.t0 = stim_events.starts(trial_idx);
3 relative_events = bml_annot_t0(cfg, all_events);
4
5 % Use roi.starts as reference (common in per-file analysis)
6 cfg = []; cfg.roi = roi(file_idx,:);
7 relative = bml_annot_t0(cfg, events_in_file);

```

0.19.2 bml_annot_rowbind — Vertical Concatenation

Concatenates multiple annotation tables, conforming all to the schema of the first non-empty table (fills missing columns with appropriate empty values).

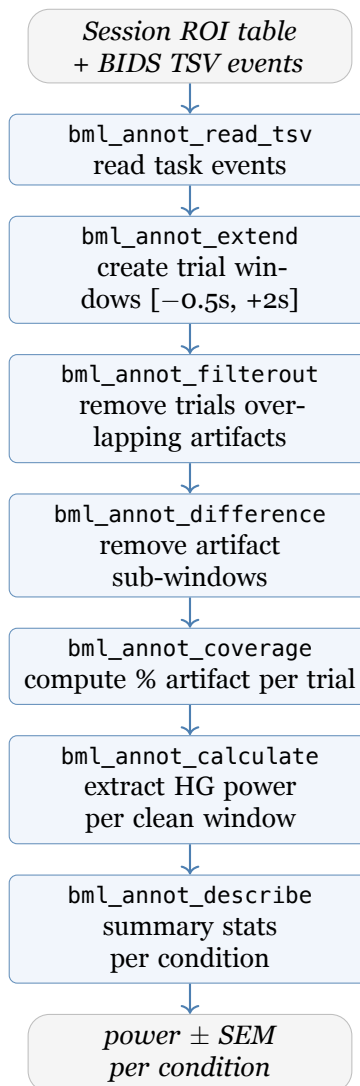
```

1 % Combine sync results from three methods
2 all_sync = bml_annot_rowbind(sync_roi_analog, sync_roi_no, sync_roi_ptb);
3
4 % Safer than [A; B; C] because it handles schema mismatches gracefully
5 % (missing columns are filled with NaN/empty)

```

0.20 Complete Worked Example: From Raw Events to Clean Trial Features

This example walks through a complete analysis pipeline using all the major annotation operations. Goal: extract high-gamma power from clean trial windows, per condition.



```

1 %% STEP 1: Load session ROI and build sync
2 roi = bml_roi_table(bml_info_raw(struct('folder', PATH_DATA)));
3 sync_roi = load(fullfile(PATH_SYNC, 'sync_roi_final.mat')); % pre-computed
4
5 %% STEP 2: Read task events (BIDS TSV)
6 events = bml_annot_read_tsv( ...
7     'sub-DM1001_ses-intraop_task-speech_events.tsv');
8 % events.starts = master-clock time (after sync applied at recording)
9 % events.trial_type = {'go', 'nogo', 'go', ...}
10
11 %% STEP 3: Create trial analysis windows
12 % Each trial = 0.5s before onset to 2s after
13 trial_windows = bml_annot_extend(events, 0.5, 2.0);
14 fprintf('Total trials: %d\n', height(trial_windows));
15
16 %% STEP 4: Load artifact annotations
17 artifacts = bml_annot_read_tsv('artifacts.tsv');
18 artifacts = bml_annot_filter(artifacts, session_window); % keep session-range only
19
20 %% STEP 5: Remove trials that are >50% artifact
21 cfg_filt = []; cfg_filt.overlap = 0.5;
22 clean_trials = bml_annot_filterout(cfg_filt, trial_windows, artifacts);

```

```

23 fprintf('Trials after artifact rejection: %d / %d\n', ...
24     height(clean_trials), height(trial_windows));
25
26 %% STEP 6: Compute per-trial artifact coverage
27 cov = bml_annot_coverage(artifacts, clean_trials);
28 clean_trials.artifact_pct = cov.coverage * 100;
29
30 %% STEP 7: Get clean (non-artifact) sub-windows for each trial
31 clean_windows = bml_annot_difference(clean_trials, artifacts);
32 % clean_windows.x_id_ links back to original trial id
33
34 %% STEP 8: Transfer trial metadata to clean windows
35 cfg_tr = []; cfg_tr.select = {'trial_type', 'value', 'session_id'};
36 clean_windows = bml_annot_transfer(cfg_tr, clean_windows, clean_trials);
37
38 %% STEP 9: Compute high-gamma power in each clean window
39 hg_roi = sync_roi(strcmp(sync_roi.filetype, 'trellis') & ...
40     strcmp(sync_roi.chantype, 'hg'),:);
41
42 log_pwr = @(raw) 10*log10(mean(raw.trial{1}(1,:).^2) + eps);
43
44 cfg_calc = []; cfg_calc.roi = hg_roi;
45 results = bml_annot_calculate(cfg_calc, clean_windows, ...
46     'log_power_hg', log_pwr);
47
48 %% STEP 10: Descriptive stats per condition
49 cfg_desc = []; cfg_desc.groupby = 'trial_type';
50 stats = bml_annot_describe(cfg_desc, results);
51 disp(stats); % trial_type | variable | mean | std | count | ...
52
53 %% STEP 11: Visualize
54 cfg_plot = []; cfg_plot.y = 'trial_type'; cfg_plot.facet = 'session_id';
55 bml_annot_plot(cfg_plot, clean_trials);
56 title('Clean trials by condition');

```

Listing 3: Complete pipeline: raw events to trial features

0.21 Annotation Algebra: Complete Reference Table

Function	Operation	What it does	Typical use
bml_annot_table	Constructor	Enforce schema, sort by starts, recompute duration	Create any annotation table
bml_roi_table	Constructor	Extend with file metadata + (s1,t1,s2,t2)	Build file/session index
bml_annot_read_tsv	I/O	Load BIDS TSV event files	Load task event logs
bml_annot_write_tsv	I/O	Save as BIDS TSV	Export synced events
bml_event2annot	Conversion	FieldTrip events → annotation table	Convert Ripple events
bml_annot2raw	Encoding	Annotations → binary/count signal	Cross-correlation, QC

Function	Operation	What it does	Typical use
bml_raw2annot	Encoding	Raw struct $\rightarrow$ time metadata	Extract file coordinates
bml_annot_extend	Dilation/Erosion	Expand or shrink interval boundaries	Analysis windows, coarsetol
bml_annot_filter	Selection	Keep rows that overlap a window	Restrict to session/ROI
bml_annot_filterout	Exclusion	Remove rows that overlap a window	Artifact rejection
bml_annot_intersect	Intersection	Clip to overlap region, track parent IDs	Find overlapping epochs
bml_annot_union	Union	Merge overlapping/touching intervals	Combine artifact lists
bml_annot_difference	Subtraction	Remove y from x , split intervals	Clean trial windows
bml_annot Consolidate	Grouping	Merge by custom criterion function	Run-length, batch, overlap
bml_annot_blocks	RLE	Contiguous same-label runs + transitions	Behavioral state segmentation
bml_annot_shadow	Gap filling	Annotations in the gaps between events	Pre/post-stimulus baseline
bml_annot_coverage	Measurement	Fraction of y covered by x	Artifact quality metric
bml_annot_overlap	Validation	Find conflicting (overlapping) pairs	Schema validation
bml_annot_left_join	Join	Key-based relational join	Add metadata columns
bml_annot_transfer	Temporal join	Copy columns based on time overlap	Assign trial labels to spikes
bml_annot_calculate	Aggregation	Apply functions to raw data per epoch	Feature extraction
bml_annot_describe	Statistics	Descriptive stats per group	Summarize features
bml_annot_detect	Detection	Threshold-crossing event detection	Detect HG bursts, peaks
bml_annot_match	Template match	Cross-correlation peak detection	Find pattern occurrences
bml_annot_t0	Translation	Shift all times by a constant	Event-relative time axis
bml_annot_rowbind	Concatenation	Vertical concat with schema conforming	Merge tables from loops
bml_annot_conform_to	Schema	Align column schema to template	Pre-concatenation fix
bml_annot_rename	Projection	Rename columns	Column name cleanup
bml_annot_sample	Sampling	Random row selection	Subsampling for tests
bml_annot_plot	Visualization	Horizontal bar plot, faceted	Inspect epoch tables

.1 Interval Algebra: Mathematical Summary

Notation

$$A = [a_1, a_2], B = [b_1, b_2], |A| = a_2 - a_1.$$

Overlap condition

$$A \cap B \neq \emptyset \Leftrightarrow a_1 < b_2 \text{ and } a_2 > b_1$$

Intersection

$$A \cap B = [\max(a_1, b_1), \min(a_2, b_2)], \text{ valid when non-empty.}$$

Union (two intervals)

$$A \cup B = [\min(a_1, b_1), \max(a_2, b_2)] \text{ when they overlap or touch } (b_1 \leq a_2 + \varepsilon).$$

Difference

$A \setminus B$: the part of A not covered by B . May produce 0, 1, or 2 intervals.

Extension/Erosion

$$\text{extend}(A, e_1, e_2) = [a_1 - e_1, a_2 + e_2]. \text{ Positive = dilate; negative = erode.}$$

Coverage

$$\text{cov}(X, y) = \frac{\sum_i |X_i \cap y|}{|y|}, \text{ where } X = \{X_i\} \text{ is a set of intervals.}$$

Index $\leftrightarrow$ time (ROI coordinates)

$$F_{s,\text{eff}} = \frac{s_2 - s_1}{t_2 - t_1}, \quad t(i) = \frac{i}{F_{s,\text{eff}}} - \frac{0.5}{F_{s,\text{eff}}} + \frac{s_2 t_1 - t_2 s_1}{s_2 - s_1}, \quad i(t) = \text{round}\left(\frac{t_2 s_1 - s_2 t_1 + (s_2 - s_1)t}{t_2 - t_1}\right)$$

Part II

The Synchronization Pipeline

BML Toolbox

Synchronization & Annotation System

Complete technical guide — Brain Modulation Laboratory

Brain Modulation Laboratory

Massachusetts General Hospital · University of Pittsburgh

April 2026

Covers: ROI tables · analog sync (Ripple/Zoom) · event sync (NeuroOmega, Psychtoolbox)
Old vs. new DP matching · consolidation · glossary · complete worked examples

.2 Glossary of Key Terms

This section defines every term used throughout the document. Read this first; refer back to it any time a term is unclear.

Annotation table

A standard MATLAB table with mandatory columns `id`, `starts`, `ends`, `duration`. The universal data structure in BML for representing any time-stamped event or segment — stimulus onsets, recording intervals, sync events, etc. Created and enforced by `bml_annot_table()`.

ROI table (Region of Interest)

An annotation table extended with file-level columns: `folder`, `name`, `nSamples`, `Fs`, `filetype`, `chantype`, `s1`, `t1`, `s2`, `t2`. One row per file (or file chunk). This is the main carrier of synchronization state in the pipeline. Created by `bml_roi_table()`.

Master / Slave

The **master** device is the one whose clock defines “ground truth” time for the entire session. In BML, this is always the **Ripple/Trellis** neural recorder. All other devices are **slaves**: their sample indices must be mapped to master-clock times. The master’s `warpfactor` is always 1.000000 (it is its own reference).

s1, t1, s2, t2 (sync coordinates)

Four numbers stored in every ROI table row that define a linear mapping from the file’s sample indices to absolute master-clock times. `s1` and `s2` are two sample indices; `t1` and `t2` are the master-clock *midpoint times* of those samples. Together they encode both a time offset and a clock-rate correction. Before synchronization, `t1/t2` come from OS timestamps (unreliable). After synchronization, they are true master-clock values.

Sample midpoint time

A sample at index i represents the signal averaged over a window of width $1/F_s$. Its “time” is the midpoint of that window. If a file starts at t_{start} , the midpoint of its first sample is $t_{\text{start}} + 0.5/F_s$. This $\pm 0.5/F_s$ correction appears everywhere in BML and is easy to overlook.

Warp factor

The ratio between the slave device’s nominal sampling rate and its effective rate as measured against the master clock. `warpfactor` > 1 means the slave clock ran *slow* (fewer samples per second than nominally); `warpfactor` < 1 means it ran *fast*. Stored per file in the ROI table. Computed as $1/w_{s1}$ from the time-warp model. Typical values deviate from 1 by 1–10 ppm.

Clock drift

The phenomenon where two independent crystal oscillators nominally running at the same frequency (e.g., 30 000 Hz) actually run at slightly different rates. A drift of 10 ppm means that after one hour, the two clocks differ by 36 ms. Drift is corrected by the `ws1` parameter in `bml_timewarp`, or by the `warpfactor` in event-based sync.

Chunk

A time window (defined in master time) used to subdivide a recording session for synchronization. Each chunk is synced independently, producing one row per (file $\times$ chunk) in the preliminary sync ROI. Chunking serves two purposes: (1) it limits memory usage by loading only part of each file at once, and (2) it provides multiple independent alignment estimates for the same file, which can be validated against each other during consolidation. Chunks are defined by `bml_chunk_sessions()`.

Consolidation

The process of merging multiple sync estimates (from different chunks of the same file, or from adjacent files of the same type) into a single, validated linear mapping per file. Think of it as fitting a “best line” through all the individual sync results and checking that they agree. If they do (residuals < 1 ms), the line is accepted. If they don’t, an error is raised. Performed by `bml_sync_consolidate()`. See Section .14 for details.

Contiguous files

Two or more files from the same device that cover consecutive time intervals with no gap (e.g., a Zoom recorder that auto-splits into hourly files). Contiguous files share the same physical clock and can be consolidated into a single linear mapping. The boundary between them is set at the midpoint of their adjacent anchor times.

Sync channel

The physical signal used to establish synchronization. For analog sync, this is a channel present in both the master and slave recordings (e.g., a click track on Trellis `ainp1` and Zoom audio). For event sync, this is a set of discrete events simultaneously visible on both devices (e.g., TTL pulses on Trellis digital input and NeuroOmega digital input).

Envelope alignment

The coarse synchronization stage in `bml_sync_analog`. Both signals are processed through the analytic signal envelope (absolute value of Hilbert transform, downsampled to 100 Hz) before cross-correlation. This produces a smooth, low-frequency representation that is robust to large time offsets (± 300 s) and polarity inversions, but has limited timing resolution (~ 10 ms).

LPF alignment

The fine synchronization stage in `bml_sync_analog`. Both signals are low-pass filtered (4th-order zero-phase Butterworth, cutoff ≤ 4000 Hz) and cross-correlated within a narrow scan window (± 1 s). This achieves sub-millisecond timing resolution but requires the coarse alignment to already be within the scan window.

bml_timealign

Low-level function that finds the time offset between two continuous signals by cross-correlation. Used inside `bml_timewarp` (and by extension `bml_sync_analog`) for both the envelope and LPF stages. Returns `slave_delta_t` and `max_corr`.

bml_timewarp

Low-level function that finds both the time offset *and* clock drift between two continuous signals by first calling `bml_timealign` and then running a non-linear optimization (Nelder-Mead simplex) to maximize signal correlation under a linear time-warp model. Returns new sync coordinates (`s1`, `t1`, `s2`, `t2`, `wt0`, `ws1`).

bml_timealign_annot

Event-based alignment function. Given two sets of discrete events (as annotation tables), finds the time shift (and optionally warp) that minimizes the sum of mismatches between slave events and their nearest master counterparts. Uses brute-force scan + Nelder-Mead refinement. Used by `audio` and `NeuroOmega sync`.

bml_sync_match_events (DP matching)

Event-*pairing* function using dynamic programming. Unlike `bml_timealign_annot` (which assumes most events are paired), this function handles sequences where events may be missing on either side. It finds the optimal subset of paired events that maximizes a weighted similarity score. Used by `bml_sync_digital` for Psychtoolbox/task laptop sync. See Section .10 for a detailed comparison with the older approach.

BIDS TSV format

The Brain Imaging Data Structure tab-separated format for event logs. Columns: onset (seconds from session start) and duration. BML reads these via `bml_annot_read_tsv()`, which auto-renames onset $\rightarrow$ starts and computes ends = starts + duration.

Trellis / Ripple

The master neural recorder used in the lab. Trellis is the software; Ripple is the hardware brand. The Ripple system records ECoG, LFP, and analog/digital auxiliary channels. Its file format (`.nev` for events, `.ns*` for continuous data) is read by the NPMK library. In BML, `filetype = 'trellis'`.

NeuroOmega

A microelectrode recording system (Alpha Omega) used for intraoperative MER recordings. Records neural signals and digital events in proprietary .mat files. Synchronized to Trellis via shared digital trigger pulses using `bml_sync_neuroomega_event()`.

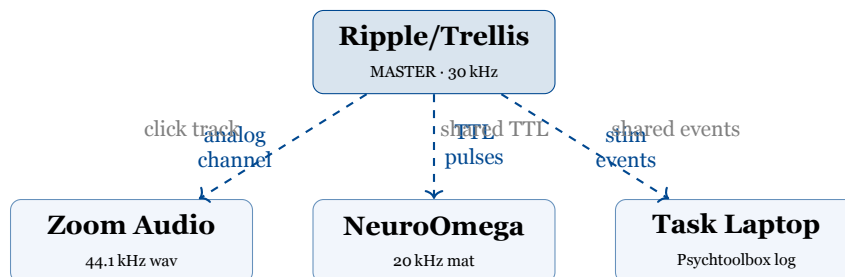
Zoom

A portable audio recorder (Zoom H-series) used to capture speech and a sync click track. Synchronized to Trellis either via a shared analog channel (`bml_sync_analog`) or via peak detection (`bml_sync_audio_event`).

.3 System Overview and Data Flow

.3.1 The synchronization problem

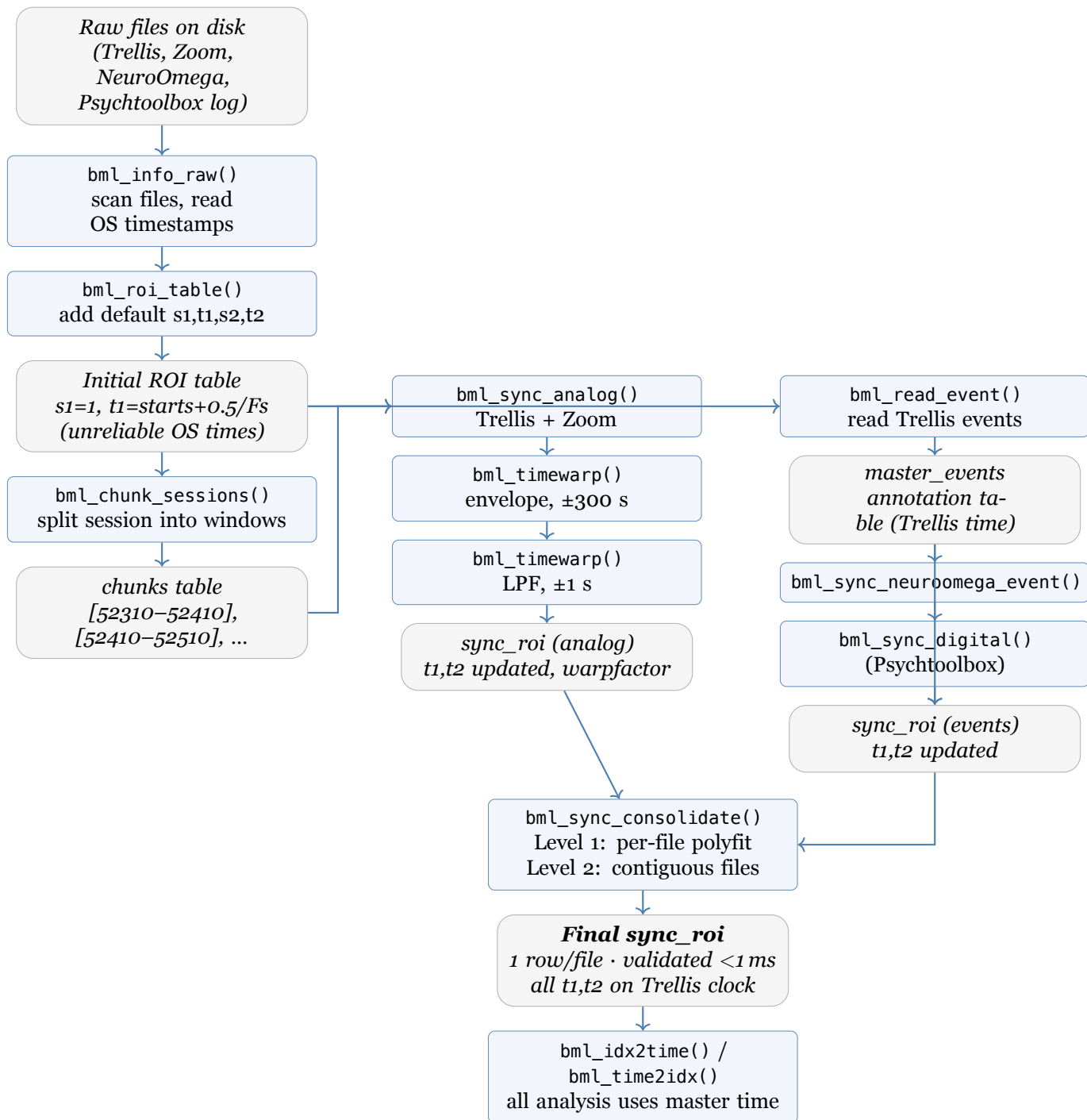
A BML recording session involves up to four independent devices, each with its own clock:



Each slave device's clock differs from the master by an **offset** (δt , seconds to minutes) and **drift** (ξ , 1–10 ppm). Both are corrected by a linear model: $t_{\text{master}} = \delta t + \xi \cdot t_{\text{slave}}$.

.3.2 Complete data flow diagram

The diagram below shows every step from raw files to synchronized data. Boxes are functions; rounded boxes are data structures; arrows show data flow.



.4 Annotation Tables and the ROI Structure

.4.1 Annotation table schema

Every annotation table is created by `bml_annot_table()` which enforces the schema:

Column	Type	Description
id	integer	Auto-assigned by sorting on starts. Not stable across calls.
starts	double (s)	Start time in seconds from midnight
ends	double (s)	End time in seconds from midnight
duration	double (s)	Always ends - starts; never set manually
<i>user cols</i>	any	Preserved after the core four

.4.2 ROI table: the sync state carrier

Column	Type	Description
folder, name	string	File location
nSamples, Fs	num	File length and nominal sample rate
filetype	string	'trellis', 'zoom', 'neuroomega', ...
chantype	string	'analog', 'digital', 'all'
s1	integer	First reference sample index
t1	double (s)	Master-clock midpoint time of sample s1
s2	integer	Second reference sample index
t2	double (s)	Master-clock midpoint time of sample s2
warpfactor	double	$1/ws_1$; 1 = no drift
chunk_id	integer	Which chunk produced this row
sync_type	string	'master' or 'slave'

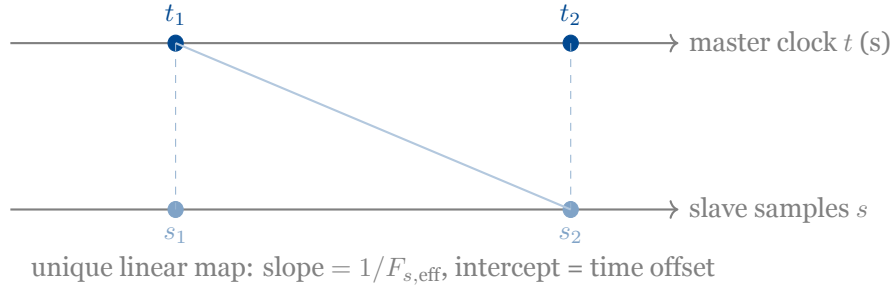
.4.3 Key annotation manipulation functions

Function	What it does
bml_annot_table(x)	Constructor. Enforces schema, sorts, assigns id, computes duration.
bml_annot_overlap	Returns all row pairs with overlapping intervals.
bml_annot_intersect	Set intersection: intervals where x and y overlap, merged columns.
bml_annot_union	Concatenate and consolidate touching/overlapping intervals.
bml_annot_difference	Subtract y's time regions from x.
bml_annot_filter	Keep only rows of a that intersect filter f.
bml_annot Consolidate	Merge contiguous/overlapping rows (run-length encode).
bml_annot_extend	Shift starts -= ext1, ends += ext2 (temporal buffers).
bml_annot_read_tsv	Read BIDS TSV event logs; renames onset→starts.
bml_roi_confluence	Set midpoint junction times between adjacent file chunks.

.5 The Coordinate System

.5.1 Why two points?

A single time offset δt corrects when a recording started but not how fast its clock ran. Two reference points (s_1, t_1) and (s_2, t_2) encode *both*: the slope of the line through them gives the effective rate; the intercept gives the offset.



[

Index-to-time (bml_idx2time)] $F_{s,\text{eff}} = (s_2 - s_1)/(t_2 - t_1)$ $t(i) = i/F_{s,\text{eff}} - 0.5/F_{s,\text{eff}} + (s_2 t_1 - t_2 s_1)/(s_2 - s_1)$

[

Time-to-index (bml_time2idx)] $i = \text{round}((t_2 s_1 - s_2 t_1 + (s_2 - s_1)t)/(t_2 - t_1))$

Before sync: $F_{s,\text{eff}} = F_s$ (nominal), intercept from OS timestamp (seconds off).

After sync: t_1, t_2 are true master-clock values; $F_{s,\text{eff}}$ may differ slightly from F_s .

.6 Building the Initial ROI Table

```

1 % bml_info_raw scans the directory and reads OS file metadata
2 cfg = [];
3 cfg.folder = '/data/patient_001/session_01';
4 cfg.filetype = {'trellis','zoom','neuroomega'};
5 info = bml_info_raw(cfg);
6 % info: starts = file_modification_time - nSamples/Fs
7 %         ends  = file_modification_time (from OS, unreliable ±seconds)
8
9 % bml_roi_table adds default sync coordinates
10 roi = bml_roi_table(info);
11 % s1 = 1
12 % t1 = starts + 0.5/Fs (midpoint of first sample -- from OS time)
13 % s2 = nSamples
14 % t2 = ends - 0.5/Fs (midpoint of last sample -- from OS time)
15 % NOTE: t1 and t2 are WRONG until synchronization is done.
```

Listing 4: Step 1: scan files and build the initial ROI

```

1 session = bml_annot_table(table(52310, 52610, 'VariableNames', {'starts','ends'}));
2 session.session_id = 1;
3
4 chunks = bml_chunk_sessions(session, 3); % split into 3 equal chunks
5 % OR: bml_chunk_sessions(session, 52450) % split at a specific time
```

```

6 % OR: bml_chunk_sessions(session, [], 60) % split into 60-second pieces
7
8 % chunks is an annotation table with columns:
9 %   id, starts, ends, duration, session_id, session_part

```

Listing 5: Step 2: define session chunks

.7 Synchronizing Audio (Zoom) to Ripple/Trellis

.7.1 Approach A: Shared analog channel (bml_sync_analog)

Both Trellis and Zoom record the same click track on an analog channel. `bml_sync_analog` finds the time shift and drift by cross-correlating these signals.

```

1 sync_channels = table( ...
2   {'trellis'; 'zoom'}, {'ainp1'; 'Ch1'}, {'analog'; 'analog'}, ...
3   'VariableNames', {'filetype', 'channel', 'chantype'});
4 sync_channels.threshold = [NaN; NaN]; % NaN = no saturation zeroing
5
6 cfg_analog = [];
7 cfg_analog.roi          = roi(ismember(roifiletype,{'trellis','zoom'}), :);
8 cfg_analog.chunks       = chunks;
9 cfg_analog.master_filetype = 'trellis';
10 cfg_analog.sync_channels = sync_channels;
11 cfg_analog.timewarp      = true;
12 cfg_analog.env_scan     = 300; % coarse: ±300 s envelope xcorr
13 cfg_analog.lpf_scan     = 1;  % fine:  ±1 s LPF xcorr
14 cfg_analog.lpf_max_freq = 4000;
15 cfg_analog.chunk_extend = 5;  % load 5 s extra each side
16 sync_roi_analog = bml_sync_analog(cfg_analog);

```

Inside `bml_sync_analog`: per-chunk pipeline

For each chunk k , for each slave filetype:

1. **Load** master sync channel (ainp1) and slave sync channel (Ch1) as `FT_DATATYPE_RAW`.
2. **Coarse alignment** — envelope warp (lines 322–327):

```

1 cfg.method = 'envelope'; cfg.env_freq = 100; cfg.scan = 300;
2 cfg.penalty_wt0_min = 1e-3; cfg.penalty_ws1 = 1e-3;
3 wc_env = bml_timewarp(cfg, master, slave);
4 slave.time{1} = bml_idx2time(wc_env, 1:length(slave.time{1}));

```

3. **Fine alignment** — LPF warp (lines 338–348):

```

1 cfg.method = 'low-pass-filter'; cfg.scan = 1;
2 cfg.lpf_freq = min([master.fsamples, slave.fsamples, 4000]);
3 cfg.penalty_wt0_min = 1e-6; cfg.penalty_ws1 = 1e-4;
4 wc_lpf = bml_timewarp(cfg, master, slave);
5 slave.time{1} = bml_idx2time(wc_lpf, 1:length(slave.time{1}));

```

4. **Write output row** (lines 352–367):

```

1 row.t1      = bml_idx2time(wc_lpf, slave_map.raw1); % true master-clock
   times

```

```

2 row.t2      = bml_idx2time(wc_lpf, slave_map.raw2);
3 row.starts  = row.t1 - 0.5 ./ row.Fs;
4 row.ends    = row.t2 + 0.5 ./ row.Fs;
5 row.warpfactor = wc_env.ws1 * wc_lpf.ws1; % combined warp from both stages

```

.7.2 Approach B: Digital triggers via audio-peak detection (bml_sync_audio_event)

Approach A requires both devices to record the same *continuous* analog waveform. Approach B does not: Ripple/Trellis logs the TTL click pulses as **digital events** (integers in its event channel), and the Zoom recorder captures the same pulses acoustically as amplitude peaks. `bml_sync_audio_event` detects those peaks with `findpeaks` and matches them to the Trellis digital event log. This is functionally the *digital* counterpart of Approach A, and is labelled `sync_channel='digital'` in the output table.

Note

When to use each approach. Use Approach A (analog) when both devices record the sync channel as a waveform: it is more robust because it exploits the full signal shape, not just peak positions. Use Approach B (digital peaks) when the Zoom recording only carries a click track and there is no shared analog channel on Trellis — the only link is the TTL events on the Ripple digital input and the acoustic peaks in the Zoom audio.

```

1 % master_events come from Ripple's DIGITAL event channel (TTL pulses)
2 master_events = bml_read_event(trellis_roi);
3 master_events = bml_event2annot([], master_events);
4 % Keep only click/sync events (e.g. value==1)
5 master_events = master_events(master_events.value == 1, :);
6
7 cfg = [];
8 cfg.roi      = roi(strcmp(roi.filetype, 'zoom'), :);
9 cfg.master_events = master_events; % <-- digital events from Ripple
10 cfg.scan     = 100;    cfg.scan_step = 0.1;
11 cfg.min_rph  = 0.5;    % peaks must be > 50% of file max amplitude
12 cfg.min_ipi  = 0.05;   % peaks must be > 50 ms apart
13 cfg.timewarp = false;
14 cfg.diagnostic_plot = true;
15 sync_roi_zoom = bml_sync_audio_event(cfg);
16 % sync_roi_zoom.sync_channel == 'digital'

```

Per file: `findpeaks(abs(audio))` → build slave event table → `bml_timealign_annot` → apply shift/warp to `t1, t2`. Contiguous files averaged together.

.8 Synchronizing NeuroOmega to Ripple/Trellis

NeuroOmega shares TTL pulse events with Trellis via a digital input. Both devices log the same physical pulses in their own clocks.

```

1 master_events = bml_read_event(trellis_roi);
2 master_events = bml_event2annot([], master_events);
3 master_events = master_events(strcmp(master_events.type, 'STIM_ON'), :);
4
5 cfg = [];
6 cfg.roi      = roi(strcmp(roi.filetype, 'neuroomega'), :);

```

```

7 cfg.master_events = master_events;
8 cfg.scan          = 100;   cfg.scan_step = 0.1;
9 cfg.coarsetol      = 100;   % restrict master events to ±100 s window
10 cfg.timewarp       = false; % true for sessions > 30 min
11 cfg.timetol        = 1e-6;
12 cfg.diagnostic_plot = true;
13 sync_roi_no = bml_sync_neuroomega_event(cfg);

```

Per file:

1. `bml_read_event(roi(i,:))` — read NeuroOmega events
2. `bml_event2annot()` — convert to annotation table
3. `bml_annot_filter(master, bml_annot_extend(roi(i,:), coarsetol))` — narrow master to ± 100 s window
4. `bml_timealign_annot(cfg, master_events, slave_events)` — find δt (and optionally ξ)
5. Update `t1, t2` via: `roi.t1 = (roi.t1 - midpt) .* warpfactor + midpt + slave_dt`

.9 Synchronizing Task Laptop (Psychtoolbox) to Ripple

.9.1 The setup

A task laptop runs Psychtoolbox (or PsychoPy) and logs every stimulus onset, response, and event with its own system clock. The same task sends TTL pulses to both the Ripple digital input and the task laptop (or the laptop time-stamps events that coincide with Ripple-logged pulses). The goal is to map task laptop event times onto the Trellis master clock.

.9.2 Step 1: Read the task log file

Task logs are typically saved as TSV or CSV files following the BIDS convention:

```

1 % BIDS TSV format: columns 'onset' and 'duration' (both in seconds from task start)
2 % bml_annot_read_tsv auto-renames 'onset' -> 'starts', computes 'ends'
3 ptb_events = bml_annot_read_tsv('sub-DM1001_ses-intraop_task-speech_events.tsv');
4 % ptb_events: id, starts, ends, duration, trial_type, response_time, ...
5 % starts = seconds from task start (laptop clock, NOT master clock)

```

Listing 6: Reading a Psychtoolbox / BIDS event log

For non-BIDS formats, build the annotation table manually:

```

1 raw = readtable('psychtoolbox_log.csv');
2 % raw has columns: onset_time, event_type, trial_id
3
4 ptb_events = bml_annot_table(table( ...
5     raw.onset_time, ...      % starts (laptop clock seconds)
6     raw.onset_time, ...      % ends (instantaneous events)
7     'VariableNames', {'starts', 'ends'}));
8 ptb_events.type = raw.event_type;
9 ptb_events.value = ones(height(ptb_events), 1);

```

.9.3 Step 2: Get the same events from Ripple

The Ripple digital input recorded the same TTL pulses that the laptop sent (or received). Extract them:

```
1 master_events = bml_read_event(trellis_roi);
2 master_events = bml_event2annot([], master_events);
3 % Keep only the event type corresponding to task triggers
4 master_events = master_events(master_events.value == 1, :);
5 % master_events.starts = times in TRELLIS clock [seconds]
```

.9.4 Step 3: Synchronize with bml_sync_digital

```
1 cfg = [];
2 cfg.timetol      = 1e-3; % 1 ms tolerance
3 cfg.sim_threshold = 0.9; % minimum similarity to accept a match
4 cfg.diagnostic_plot = true;
5 cfg.plot_title   = 'Psychtoolbox to Trellis';
6
7 % Weight configuration for event matching
8 cfg.timetol      = 1e-3; % timing tolerance for similarity kernel
9 cfg.weight_time_pre = 1/4; % weight for pre-event interval match
10 cfg.weight_time_post = 1/4; % weight for post-event interval match
11 cfg.weight_value_pre = 1/4; % weight for pre-event value match
12 cfg.weight_value_post = 1/4; % weight for post-event value match
13 cfg.weight_onset     = 0; % set > 0 to prefer temporally close matches
14
15 [sync_roi_ptb, hF] = bml_sync_digital(cfg, master_events, ptb_events);
16 % sync_roi_ptb columns:
17 % starts, ends -- time window of matched events (in master time)
18 % delta_t      -- shift to add to laptop times to get master times
19 % warpfactor    -- clock rate correction (1 = no drift)
20 % mean_sim      -- mean similarity score of matched event pairs
21 % s1, t1, s2, t2 -- sync coordinates (from sample column if available)
```

Listing 7: Synchronizing Psychtoolbox events to Trellis

.9.5 Step 4: Apply sync to task events

After synchronization, convert all laptop event times to master clock:

```
1 % Apply the linear transformation from sync_roi_ptb
2 delta_t = sync_roi_ptb.delta_t(1);
3 warpfactor = sync_roi_ptb.warpfactor(1);
4 tbar      = mean(ptb_events.starts); % pivot for warping
5
6 ptb_events_synced = ptb_events;
7 ptb_events_synced.starts = (ptb_events.starts - tbar) .* warpfactor + tbar +
    delta_t;
8 ptb_events_synced.ends   = (ptb_events.ends   - tbar) .* warpfactor + tbar +
    delta_t;
9 % Now ptb_events_synced.starts are in TRELLIS (master) clock seconds
```

.9.6 Step 4b: Exact-match fallback strategy

`bml_sync_digital` returns a single linear model. A more precise strategy, used in production scripts, exploits the DP matching directly:

- For task events that **matched a Ripple event** in the DP: substitute the exact Ripple timestamp. No interpolation error.
- For task events in **gaps** (no matching Ripple event — task events the Ripple didn't log): use the linear warp as a fallback.

This hybrid approach gives sub-millisecond accuracy for most events while gracefully handling trials where the Ripple event was not recorded.

```

1 % SYNCHRONIZING EVENT FILES TO RIPPLE EVENTS
2 % Will inherit exact timing from Ripple. Looping through events_files.
3 events_files.delta_t(:) = nan;
4 events_files.warpfactor(:) = nan;
5 events_files.n(:) = nan;
6
7 for i = 1:height(events_files)
8
9     % Skip if no matching Ripple (NEV) events exist for this file
10    % (prevents matching pre-op events to intraop NEV files)
11    nev_events_files = bml_annot_filter(roi_nev, events_files(i,:));
12    if isempty(nev_events_files); continue; end
13
14    task_events = bml_annot_read_tsv( ...
15        fullfile(events_files.path{i}, events_files.filename{i}));
16
17    % DP matching: find which task events correspond to which master events
18    cfg = [];
19    cfg.timetol = 0.003; % 3 ms timing tolerance for similarity kernel
20    cfg.weight_onset = 0.1; % small onset weight to prefer temporally close
21    % matches
22    cfg.diagnostic_plot = true;
23    [idxs_master, idxs_task, mean_sim, simv] = ...
24        bml_sync_match_events(cfg, master_events, task_events);
25
26    % Find the longest contiguous block of matched pairs
27    % (guards against spurious matches at session boundaries)
28    disconts_idx = [1, find(diff(idxs_master) > 3), length(idxs_master)];
29    [~, cont_idx] = max(diff(disconts_idx));
30    start_idx = disconts_idx(cont_idx) + 1;
31    end_idx = disconts_idx(cont_idx + 1) - 1;
32
33    % Fit linear warp as FALLBACK (for events without a direct Ripple match)
34    tbar = mean(task_events.starts(idxs_task), 'omitnan');
35    p = polyfit( ...
36        task_events.starts(idxs_task(start_idx:end_idx)) - tbar, ...
37        master_events.starts(idxs_master(start_idx:end_idx)) - tbar, 1);
38
39    events_files.warpfactor(i) = p(1);
40    events_files.delta_t(i) = p(2);
41    events_files.n(i) = height(task_events(idxs_task, :));
42
43    % Compute fallback times for ALL task events via the linear model
44    task_events.starts_corrected = ...

```

```

44     (task_events.starts - tbar) .* p(1) + tbar + p(2);
45
46     % PRIMARY: substitute exact Ripple time for matched events
47     task_events.starts = bml_map( ...
48         1:height(task_events), ...
49         idxs_task, master_events.starts(idxs_master), nan);
50
51     % FALLBACK: fill gaps with linear-warp times
52     task_events.starts(isnan(task_events.starts)) = ...
53         task_events.starts_corrected(isnan(task_events.starts));
54
55     % Save synchronized event file
56     bml_annot_write_tsv(task_events, ...
57         fullfile(PATH_ANNOT, strrep(events_files.filename{i}, '.tsv', '-sync.tsv')));
58
59     fprintf('sync %s delta_t=%.4f mean_sim=%.3f warpfactor=%.7f\n', ...
60         events_files.filename{i}, p(2), mean_sim, p(1));
61 end

```

Listing 8: Fallback strategy: exact Ripple times where available, linear interpolation elsewhere

Note

Why two passes? The linear warp corrects for clock drift globally. But Ripple’s event timestamps are the *ground truth*: they were recorded at hardware precision (30 kHz sample clock). Where a direct match exists, using the Ripple timestamp is always more accurate than any interpolated value. The fallback linear model fills only the gaps.

Warning

Choosing the longest contiguous block. The DP matching may produce a few spurious matches at the very start or end of the session (where the event sequences don’t overlap well). Taking the longest contiguous block — defined by no gap of more than 3 unmatched master events — avoids these boundary artefacts in the linear fit.

.10 Event Alignment: From Brute-Force to Dynamic Programming

.10.1 The problem with simple event matching

Both `bml_sync_audio_event` and `bml_sync_neuroomega_event` use `bml_timealign_annot`, which works well when:

- Most slave events have a corresponding master event (few missing events)
- The initial OS-timestamp coarse alignment is roughly correct

But for Psychtoolbox/task events, **neither assumption may hold**:

- The laptop clock can be hours off from the master
- Some trial types may be absent on one side
- The event sequence may contain events of many different values

`bml_sync_match_events` solves this with dynamic programming.

.10.2 The old approach (archived, pre-2025)

The archived version (`bml_sync_match_events_archivePLB20250715.m`) used a **4-feature** similarity function:

```

1 % Feature vector for event i (4 features):
2 % [ interval_before_i, interval_after_i, value[i], value[i+1] ]
3 x1 = [events1_delta_starts(1:end-1), ... % pre-interval
4       events1_delta_starts(2:end),    ... % post-interval
5       events1.value(1:end-2),         ... % pre value
6       events1.value(2:end-1)];        % post value
7
8 % Similarity (old): no onset term, fixed equal weights
9 similarity_fun = @(x1,x2) ...
10    wdt1 .* (1./(1+((x1(:,1)-x2(:,1))./timetol).^2)) + ... % pre-interval
11    wdt2 .* (1./(1+((x1(:,2)-x2(:,2))./timetol).^2)) + ... % post-interval
12    wv1  .* (x1(:,3)==x2(:,3)) + ... % pre-value
13    wv2  .* (x1(:,4)==x2(:,4)); % post-value

```

Listing 9: Old 4-feature similarity (archived)

Problem: without an onset term, the algorithm had no preference for matches that are close in *absolute* time. Two events with identical inter-event intervals but 1 hour apart in the session were considered equally good candidates. In sessions with repetitive, regular event patterns, this could lead to mismatches.

.10.3 The new approach (current)

The current version adds a **5th feature**: the absolute onset time of each event. This breaks ties by preferring pairs that are close in the session:

```

1 % Feature vector for event i (5 features):
2 % [ pre-interval, post-interval, pre-value, post-value, ONSET TIME ]
3 x1 = [events1_delta_starts(1:end-1), ...
4       events1_delta_starts(2:end),    ...
5       events1.value(1:end-2),         ...
6       events1.value(2:end-1),         ...
7       events1.starts(1:end-2)]; % <-- NEW: absolute onset time
8
9 % Similarity (new): onset term with configurable weight
10 similarity_fun = @(x1,x2) ...
11    wdt1 .* (1./(1+((x1(:,1)-x2(:,1))./timetol).^2)) + ... % pre-interval
12    wdt2 .* (1./(1+((x1(:,2)-x2(:,2))./timetol).^2)) + ... % post-interval
13    wv1  .* (x1(:,3)==x2(:,3)) + ... % pre-value
14    wv2  .* (x1(:,4)==x2(:,4)) + ... % post-value
15    wo   .* (1./(1+((x1(:,5)-x2(:,5))./onsettol).^2)); % <-- onset time
16
17 % All weights are normalized to sum to 1
18 total = wdt1 + wdt2 + wv1 + wv2 + wo;
19 wdt1 = wdt1/total; % etc.

```

Listing 10: New 5-feature similarity (current)

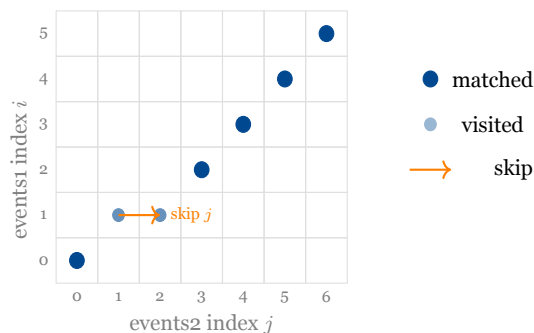
Summary of differences

Aspect	Old (archived)	New (current)
Features per event	4	5
Onset term	Absent	Present (weight_onset, default 0)
Weight params	weight_time_pre/post, weight_value_pre/post	Same + weight_onset + onsettol
Normalization	Fixed 1/4 each	Sum-normalized
Robustness	Can confuse repetitive patterns	Onset weight breaks ties
Backwards compat	—	weight_onset=0 replicates old behavior

Setting `cfg.weight_onset = 0` (the default) exactly reproduces the old behavior. Setting it to a positive value (e.g., 0.5) adds a preference for temporally proximate matches, which is useful for long sessions or repetitive event patterns.

.10.4 The dynamic programming algorithm (both versions)

The DP algorithm itself is identical in both versions — only the similarity function differs.



```

1 dp = zeros(n+1, m+1);
2 for i = 1:n
3     for j = 1:m
4         dp(i+1,j+1) = max([ ...
5             dp(i,j) + similarity_fun(x1(i,:), x2(j,:)), ... % match i to j
6             dp(i+1,j), ... % skip j (missing in
7                 events1)
8             dp(i,j+1) ]); ... % skip i (missing in
9                 events2)
10    end
11 end
12 % Backtrack from dp(n+1, m+1) to recover paired indices

```

Listing 11: DP recurrence (same in old and new versions)

This is identical in structure to the Longest Common Subsequence (LCS) algorithm, but with a continuous similarity score instead of binary match/mismatch. Skipping costs nothing (dp stays flat), while matching adds the similarity score. The backtracking recovers the set of pairs that maximises the total accumulated similarity.

.10.5 How `bml_sync_digital` uses the DP output

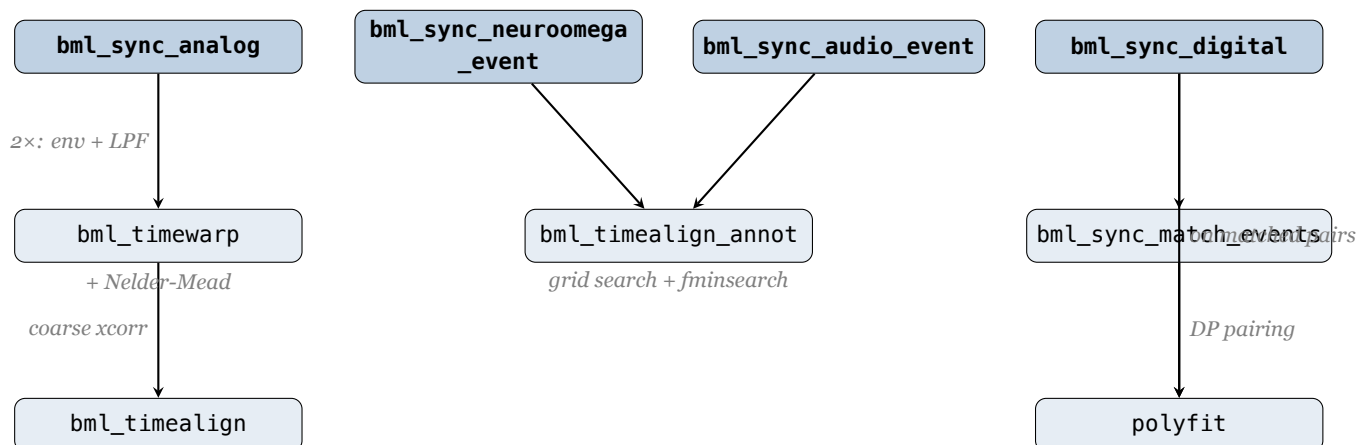
After DP matching, `bml_sync_digital` fits a linear model:

```
1 % dtv = time differences for all matched pairs
2 dtv = master_events.starts(idxs_master) - slave_events.starts(idxs_slave);
3
4 % Detect contiguous high-similarity chunks
5 % (similarity drops when event sequences diverge)
6 cfg1.threshold = sim_threshold; % default 0.9
7 chunks = bml_annot_detect(cfg1, sim_raw); % segments where sim > 0.9
8
9 % For each chunk: fit linear model
10 for i = 1:height(chunks)
11     chunk_idx = ceil(chunks.starts(i)):floor(chunks.ends(i));
12     p = polyfit(master_events.starts(idxs_master(chunk_idx)) - tbar, ...
13               dtv(chunk_idx), 1);
14     chunks.warp(i) = p(1);
15     chunks.delta_t(i) = p(2);
16 end
17
18 % Consolidate chunks with consistent delta_t
19 cfg1.criterion = @(x) (abs(max(x.delta_t) - min(x.delta_t)) < cfg1.timetol);
20 cons_chunks = bml_annot_consolidate(cfg1, chunks);
21
22 % Final polyfit across all events in each consolidated chunk
23 % If slave has a 'sample' column, also compute (s1,t1,s2,t2) coordinates:
24 p = polyfit(slave_events.sample(idxs(chunk_idx)), ...
25           master_events.starts(idxs_master(chunk_idx)), 1);
26 cons_chunks.s1(i) = slave_events.sample(idxs(chunk_idx(1)));
27 cons_chunks.t1(i) = cons_chunks.s1(i) * p(1) + p(2);
28 cons_chunks.s2(i) = slave_events.sample(idxs(chunk_idx(end)));
29 cons_chunks.t2(i) = cons_chunks.s2(i) * p(1) + p(2);
```

.11 Alignment Function Hierarchy

This section provides a complete map of the low-level alignment functions, how they relate to each other, and when each one is appropriate.

.11.1 Call graph



.11.2 The two dimensions

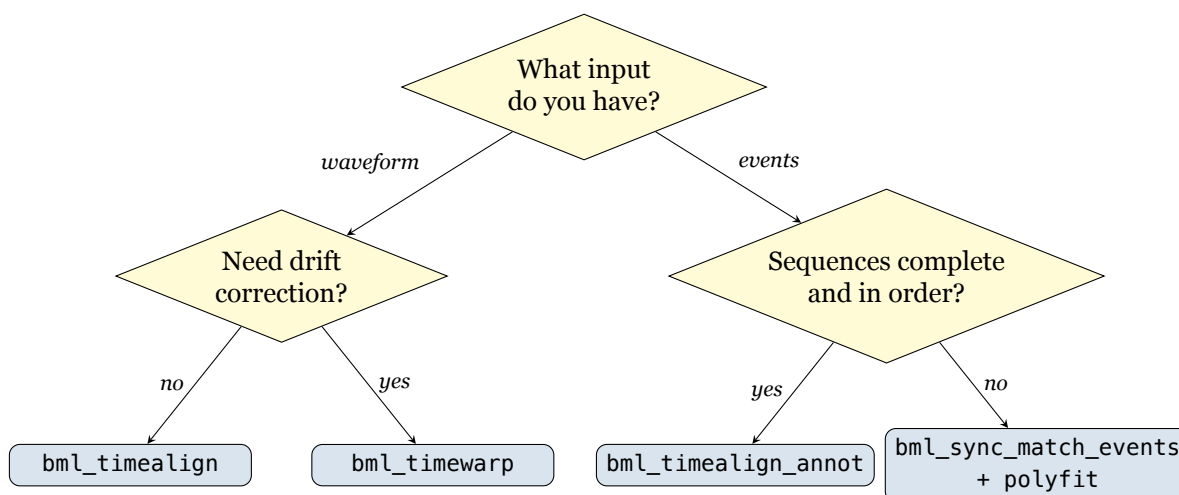
There are two independent choices:

	Waveform input	Event list input
Offset only	bml_timealign	bml_timealign_annot
Offset + drift	bml_timewarp	bml_timewarp_annot

- **Waveform:** continuous FT_DATATYPE_RAW signal (e.g. shared analog click track)
- **Event list:** annotation table of timestamps (e.g. TTL pulses, audio peaks)
- **Offset:** constant shift δt only
- **Drift:** linear clock rate $ws_1 \neq 1$ fitted in addition to offset

bml_timewarp_annot is an alias for bml_timealign_annot with timewarp=true forced. It is not called by any pipeline function — all callers use bml_timealign_annot directly.

.11.3 Decision guide: which function to call?



Situation	Call this	Why
Zoom analog click track	<code>bml_sync_analog</code>	shared waveform, xcorr + warp
Zoom audio peaks (no shared cable)	<code>bml_sync_audio_event</code>	peaks $\square$ events, then grid search
NeuroOmega TTL pulses	<code>bml_sync_neuroomega_event</code>	same signal, grid search
Psychtoolbox task events	<code>bml_sync_digital</code>	sequences may differ $\square$ DP
Custom: waveform, with drift	<code>bml_timewarp</code>	
Custom: events, known complete	<code>bml_timealign_annot</code>	
Custom: events, possibly missing	<code>bml_sync_match_events</code>	

Note

Why does NeuroOmega use `bml_timealign_annot` instead of DP? Both NeuroOmega and Ripple are wired to the same TTL cable, so every pulse recorded by one is also recorded by the other. The sequences are identical — you only need to find the time shift, not figure out which event matches which. The simpler grid-search approach is sufficient and faster.

Psychtoolbox events, by contrast, travel over a potentially unreliable channel and represent a different set of event types. The DP algorithm in `bml_sync_match_events` is needed to establish correspondence before estimating the offset.

.12 Event Alignment Internals: `bml_timealign_annot`

Used by audio and NeuroOmega sync. Finds δt (and optionally ξ) that minimises the total event mismatch.

[

$$\text{Event alignment cost] } \mathcal{C}(\delta t) = \sqrt{\sum_i \left[\min \left(\min_j |s_i + \delta t - m_j|, t_{\text{clip}} \right) \right]^2}$$

Algorithm: (1) brute-force scan on `linspace(-scan, scan, 2*scan/scan_step+1)`; (2) censor outlier events with `residual > censor_mismatch`; (3) `fminsearch` refinement in 1-D (no warp) or 2-D (with warp).

.13 Continuous Signal Alignment

.13.1 `bml_timealign` — cross-correlation

1. Compute scan range (max possible shift before files stop overlapping)
2. Pad both signals to cover the scan window; resample to `resample_freq` (10 kHz)
3. Preprocess: **envelope** (abs of Hilbert, downsample to 100 Hz) or **LPF** (4th-order Butterworth)
4. Normalize: subtract robust median, divide by robust std (16th–84th percentile)

5. `xcorr(..., 'coeff')`, find peak lag $\rightarrow$ `slave_delta_t`
6. If LPF method: also test inverted polarity; keep best

.13.2 `bml_timewarp` — linear warp optimization

[

Warp model] $w(t) = t_{\text{pivot}} + wt_0 + (t - t_{\text{pivot}}) \cdot ws_1$

[

Warp cost function] $\mathcal{L}(wt_0, ws_1) = -\frac{\langle f(w(\mathbf{t})), p \rangle}{\|f(w(\mathbf{t}_0))\| \|p\|} + \left(\frac{wt_0}{P_{\delta t}}\right)^2 + \left(\frac{ws_1 - 1}{P_{\xi}}\right)^4$

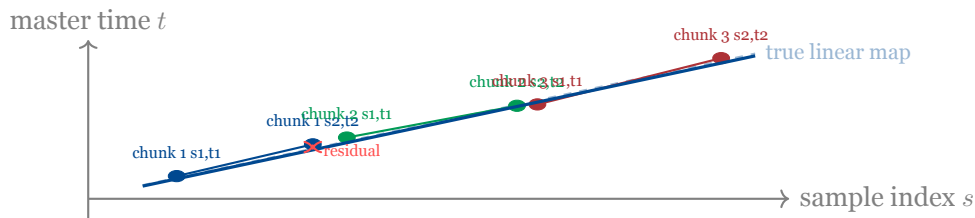
Minimised by Nelder-Mead simplex (`fminsearch`). Output: `new(s1, t1, s2, t2)` and `warpfactor = 1/ws1`.

.14 Consolidation: What It Is and Why It Matters

.14.1 The problem consolidation solves

After running `bml_sync_analog` (or any event sync), the `sync_roi` table has **multiple rows per file** — one per chunk. For example, a 5-minute Zoom recording synced in 3 chunks produces 3 rows, each with slightly different (`s1`, `t1`, `s2`, `t2`) estimates.

Before we can use these coordinates for data loading, we need to answer: do all three estimates agree? And if so, what is the *single* best linear mapping for the whole file?



Each chunk gives two anchor points (a short line segment). If all three segments lie on the same global line (residuals < 1 ms), consolidation succeeds and we replace the three rows with one, using the global best-fit line.

.14.2 Level 1: per-file consolidation

```

1 % For a file with N chunks, bml_sync_consolidate does:
2 % 1. Convert all (s1,s2) to raw (global) sample indices via s2raw()
3 i_roi = s2raw(i_roi); % adds columns raw1, raw2
4
5 % 2. Fit a single line through all anchor points
6 [p, max_delta_t, off_idx] = sync_cons_group(i_roi, timewarp, 'raw');
7 %   p(1) = 1/Fs_eff (slope), p(2) = time intercept
8 %   max_delta_t = max residual in seconds
9
10 % 3. Accept or reject
11 if max_delta_t <= timetol % default 1 ms
12   i_roi.t1 = polyval(p, i_roi.raw1);

```

```

13 i_roi.t2 = polyval(p, i_roi.raw2);
14 i_roi.warpfactor = 1 ./ (i_roi.Fs .* p(1));
15 else
16     error('consolidation failed: max residual %f ms', max_delta_t*1000);
17 end

```

.14.3 Level 2: contiguous file consolidation

For devices that split recordings into consecutive files (Zoom, NeuroOmega), the same logic is applied *across* files of the same type. The criterion for attempting this is that the files are contiguous in time ($\text{gap} < \text{timetol_contiguous}$):

```

1 % Detect contiguous file groups
2 cfg1.criterion = @(x) ...
3     (abs(sum(x.duration) - max(x.ends) + min(x.starts)) < height(x)*
4         timetol_contiguous);
5 i_roi_cont = bml_annot_consolidate(cfg1, bml_roi_confluence(i_roi));
6
7 % For each contiguous group: fit one global line across all files
8 % After fitting, set junction times at file boundaries:
9 for i = 1:(height(i_roi_cont)-1)
10     % Midpoint between adjacent anchor times = junction
11     junction = (i_roi_cont.j.t2(i) + i_roi_cont.j.t1(i+1)) / 2;
12     i_roi_cont.j.ends(i) = junction;
13     i_roi_cont.j.starts(i+1) = junction;
14 end

```

The **midpoint junction rule** ensures no gap and no overlap between adjacent files. The timing uncertainty at each file boundary is split evenly.

.14.4 What the consolidated table looks like

```

1 cfg_cons = [];
2 cfg_cons.roi = [sync_roi_analog; sync_roi_no; sync_roi_ptbl];
3 cfg_cons.timetol = 1e-3;
4 cfg_cons.timetol_contiguous = 1e-3;
5 cfg_cons.contiguous = true;
6 cfg_cons.group = 'session_id';
7 cfg_cons.plot_diagnostic = true; % shows pairwise residual matrix per file
8 sync_roi_final = bml_sync_consolidate(cfg_cons);

```

id	starts	ends	s1	t1	s2	t2	name	Fs	warpfactor	filetype
1	52310.0	52610.0	1	52310.000	9000000	52609.999	session01.nev	30000	1.000000	trellis
2	52313.2	52612.9	1	52313.201	13219590	52612.897	zoom_001-3	44100	0.999998	zoom
3	52311.7	52611.1	1	52311.702	6001200	52611.098	NO_001-2	20020	1.000003	neuroomega

Interpreting warpfactor: Zoom at 0.999998 means its clock ran 2 ppm *fast* (slightly fewer samples per real second than 44100). NeuroOmega at 1.000003 means its clock ran 3 ppm *slow*. Over a 5-minute session: Zoom off by 0.6 ms, NeuroOmega off by 0.9 ms — easily within 1 ms tolerance.

.15 Complete End-to-End Example

```

1 %% STEP 1 -- Build initial ROI
2 roi = bml_roi_table(bml_info_raw(struct('folder','/data/p001/s01')));
3
4 %% STEP 2 -- Define chunks
5 session = bml_annot_table(table(52310, 52610, 'VariableNames',{'starts','ends'}));
6 session.session_id = 1;
7 chunks = bml_chunk_sessions(session, 3); % 3 chunks of ~100 s
8
9 %% STEP 3 -- Analog sync: Trellis + Zoom
10 sync_ch = table({'trellis','zoom'},{'ainp1','Ch1'},{'analog','analog'}, ...
11               'VariableNames',{'filetype','channel','chantype'});
12 sync_ch.threshold = [NaN; NaN];
13
14 cfg_a = [];
15 cfg_a.roi = roi(ismember(roifiletype,{'trellis','zoom'}),:);
16 cfg_a.chunks = chunks; cfg_a.master_filetype = 'trellis';
17 cfg_a.sync_channels = sync_ch;
18 cfg_a.timewarp = true; cfg_a.env_scan = 300; cfg_a.lpf_scan = 1;
19 sync_roi_az = bml_sync_analog(cfg_a);
20
21 %% STEP 4 -- Extract master events from Trellis
22 master_events = bml_event2annot([], bml_read_event(roifiletype,{'trellis',
23               '}),:));
24 stim_events = master_events(strcmp(master_events.type,'STIM_ON'),:);
25
26 %% STEP 5 -- Event sync: NeuroOmega
27 cfg_no = [];
28 cfg_no.roi = roi(strcmp(roifiletype,'neuroomega'),:);
29 cfg_no.master_events = stim_events;
30 cfg_no.scan = 100; cfg_no.coarsetol = 100; cfg_no.timewarp = false;
31 sync_roi_no = bml_sync_neuroomega_event(cfg_no);
32
33 %% STEP 6 -- Event sync: Psychtoolbox
34 ptb = bml_annot_read_tsv('sub-p001_ses-intraop_task-speech_events.tsv');
35 ptb_triggers = ptb(strcmp(ptb.trial_type,'STIM_ON'),:);
36 ptb_triggers.value = ones(height(ptb_triggers),1);
37
38 trellis_triggers = stim_events;
39 trellis_triggers.value = ones(height(trellis_triggers),1);
40
41 cfg_ptb = [];
42 cfg_ptb.timetol = 1e-3; cfg_ptb.sim_threshold = 0.9;
43 cfg_ptb.weight_onset = 0.2; cfg_ptb.onsettol = 200;
44 cfg_ptb.diagnostic_plot = true;
45 [sync_roi_ptb, hF] = bml_sync_digital(cfg_ptb, trellis_triggers, ptb_triggers);
46
47 %% STEP 7 -- Consolidate everything
48 cfg_cons = [];
49 cfg_cons.roi = [sync_roi_az; sync_roi_no];
50 cfg_cons.timetol = 1e-3; cfg_cons.contiguous = true; cfg_cons.group = 'session_id';
51 sync_roi_final = bml_sync_consolidate(cfg_cons);
52 save('/data/p001/s01/sync_roi.mat','sync_roi_final');
53
54 %% STEP 8 -- Apply PTB sync and use synchronized data
55 delta_t = sync_roi_ptb.delta_t(1);

```



```

55 warpfactor = sync_roi_ptb.warpfactor(1);
56 tbar = mean(ptb_triggers.starts);
57 ptb.starts = (ptb.starts - tbar) .* warpfactor + tbar + delta_t;
58 ptb.ends = (ptb.ends - tbar) .* warpfactor + tbar + delta_t;
59 % All ptb event times are now in Trellis master clock
60
61 % Load Zoom audio for a specific time window (in master time)
62 cfg_load = [];
63 cfg_load.roi = sync_roi_final(strcmp(sync_roi_final.filetype, 'zoom'),:);
64 cfg_load.toi = [52350, 52400]; % master clock seconds
65 zoom_raw = bml_load_continuous(cfg_load);

```

Listing 12: Complete synchronization script: Trellis + Zoom + NeuroOmega + Psychtoolbox

.16 Common Pitfalls

- 1. id is not stable.** Reassigned at every `bml_annot_table()` call. Use as a within-table label only.
- 2. The $\pm 0.5/F_s$ midpoint offset.** `t1 = starts + 0.5/Fs`, not `starts`. Forgetting this causes systematic half-sample errors.
- 3. $F_{s,\text{eff}} \neq F_s$ after warping.** Always use `bml_idx2time`; never divide by `Fs` directly.
- 4. chunk_extend too small.** Set `cfg.chunk_extend` $\geq$ expected OS timestamp error (5–30 s typical). If too small, the slave signal will not overlap the master in the extended chunk.
- 5. coarsetol too small (NeuroOmega/PTB).** If OS timestamps are off by more than `coarsetol`, the correct master events are excluded from the alignment window. Increase `coarsetol` to match the expected offset.
- 6. DP returns > 10 chunks.** `bml_sync_digital` raises an error if similarity drops below `sim_threshold` more than 10 times. This usually indicates poor event matching — try adjusting weights or lowering `sim_threshold`.
- 7. Consolidation failure.** A bad chunk (one where sync was unreliable) will cause the per-file polyfit to have large residuals. Use `cfg.plot_diagnostic = true` to identify it and re-run the sync for that chunk only.
- 8. Combining analog and event sync ROI.** `bml_sync_analog` adds `chunk_id` and `sync_channel` columns; event sync functions may not. Make sure schemas match before vertically concatenating.
- 9. Warpfactor $\gg 1$ or $\ll 1$.** Values deviating from 1 by more than 10^{-4} (100 ppm) are suspicious. Check whether the sync signal was clipped, noisy, or whether the wrong channel was selected.

.17 Diagnostic Plots: How to Enable and Interpret

Every sync function exposes a diagnostic flag. Enable it during development and whenever a sync fails validation.

```
cfg.diagnostic_plot = true; % bml_sync_analog, bml_sync_neuroomega_event,
```

```

                                % bml_sync_audio_event , bml_sync_digital ,
                                % bml_sync_match_events
cfg.plot_diagnostic = true;    % bml_sync_consolidate only

```

.17.1 bml_sync_match_events — DP similarity plot

Produces a 2-panel figure.

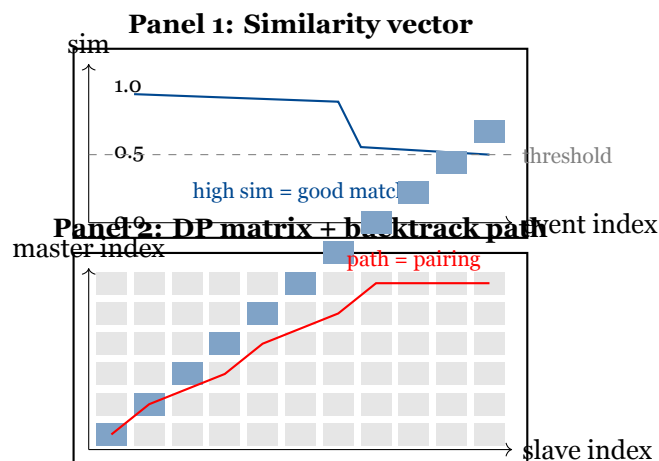


Figure 1: Schematic of `bml_sync_match_events` diagnostic figure.

Pattern	Meaning
All similarity values near 1	Perfect match
Cluster of high values then drop	One clean matched block; use <code>sim_threshold</code> to isolate it
All values < 0.5	Poor match — wrong scan range or no overlap
Backtrack path is a smooth diagonal	Clean 1:1 correspondence
Path has horizontal/vertical runs	Events missing on one side
Path stays in one corner	No real overlap — wrong files

.17.2 bml_sync_audio_event / bml_sync_neuroomega_event — event alignment plot

One 2-panel figure per file: a timeline (left) and a residual histogram (right).

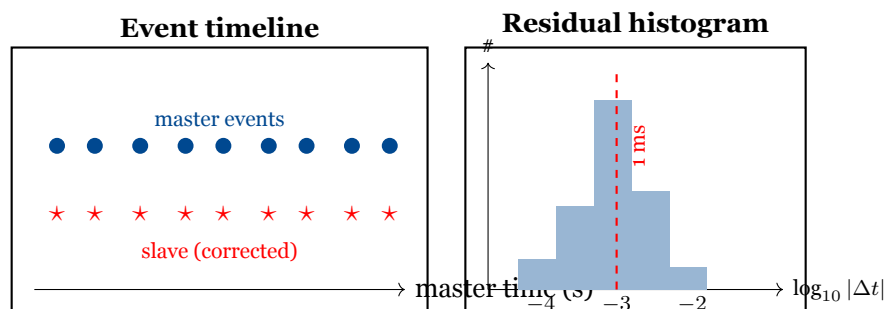


Figure 2: Schematic of event-alignment diagnostic. Good output: circles and stars overlap; histogram peak at $\log_{10} |\Delta t| \leq -3$ (i.e. ≤ 1 ms).

Pattern	Meaning
Circles and stars overlap	Good alignment
Stars consistently offset	<code>delta_t</code> estimate slightly off; widen scan
Stars spread out	Residual drift; enable <code>timewarp = true</code>
Many extra stars	False peaks; increase <code>min_rph</code>
Many circles without a star	Missed events; decrease <code>min_rph</code>
Histogram peak at ≤ -3 (≤ 1 ms)	Excellent
Peak at -2 (10 ms)	Marginal; borderline for neural data
Peak at -1 or higher	Poor; inspect timeline
Bimodal histogram	Matched and unmatched events mixed

.17.3 `bml_sync_digital` — residual + alignment plot

One 3-panel figure per contiguous matched chunk.

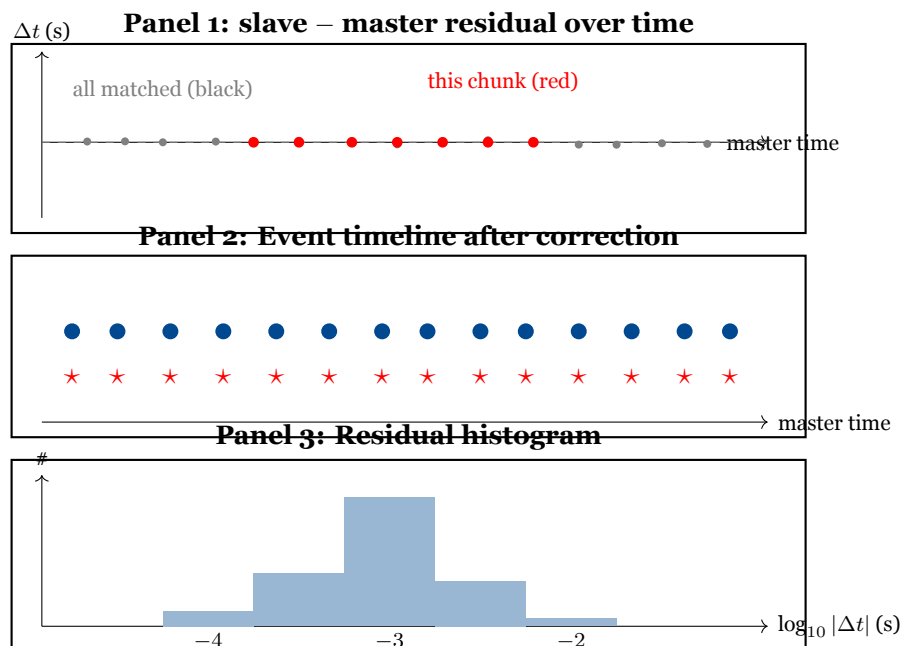


Figure 3: Schematic of `bml_sync_digital` 3-panel diagnostic. Good output: Panel 1 residuals are flat near 0; Panel 2 circles and stars overlap; Panel 3 peak at ≤ -3 .

Panel 1 interpretation:

Pattern	Meaning
Flat horizontal band near 0	Pure offset, no drift — good
Tilted straight line	Linear drift; enable <code>timewarp = true</code>
Curved or scattered	Non-linear or bad matching
Sudden jump	Two sessions mixed; check <code>cfg.group</code>

.17.4 `bml_sync Consolidate` — pairwise residual heatmap

One $N \times N$ heatmap per file (Level 1) and per contiguous group (Level 2), where N = number of chunks. Color = pairwise δt disagreement.

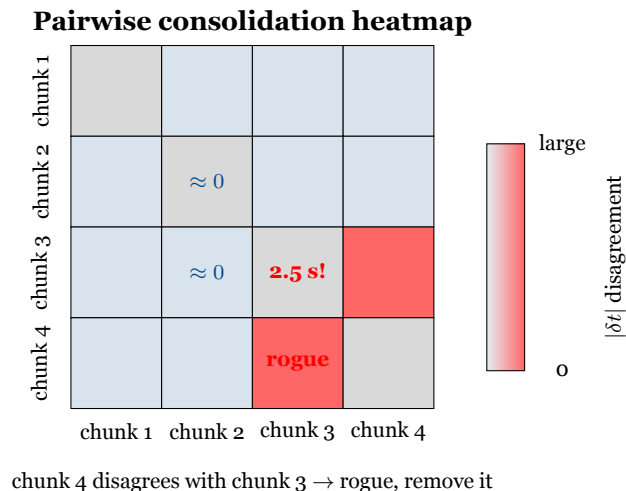


Figure 4: Schematic consolidation heatmap. Blue = good agreement; red = rogue chunk. Remove the row/column that is uniformly bright and re-run consolidation.

Pattern	Meaning
All cells dark	All chunks agree — consolidation will succeed
One row/column bright	That chunk is a rogue; remove and retry
Checkerboard	Alternating drift; shorten chunk windows
All cells bright	Sync failed entirely

Listing 13: Removing a rogue chunk and re-consolidating

```
% Find rogue chunks (>5 ms residual)
bad = sync_roi.meanerror > 0.005;
sync_roi_clean = sync_roi(~bad, :);
sync_roi_final = bml_sync Consolidate(struct('roi', sync_roi_clean));
```

.17.5 Quick-reference: what to check first

Question	Plot	Panel
Did DP find the right event pairs?	bml_sync_match_events	DP matrix path
Are residuals < 1 ms?	Any function	Residual histogram
Is there drift?	bml_sync_digital	Residual over time
Is one chunk an outlier?	bml_sync Consolidate	Pairwise heatmap
Are all events accounted for?	Audio/NeuroOmega	Timeline
Is similarity high enough?	bml_sync_match_events	Similarity vector

.18 Best Practices and Lab Conventions

.18.1 Choose your sync method by signal quality

Situation	Method	Why
Zoom + shared analog waveform on Trellis	Approach A: <code>bml_sync_analog</code>	Exploits full waveform shape; most robust
Zoom + only TTL click track	Approach B: <code>bml_sync_audio_event</code>	Audio peaks matched to Ripple digital events
NeuroOmega + shared TTL	<code>bml_sync_neuroomega_event</code>	TTL events present on both devices
Psychtoolbox / task laptop	DP matching + fallback	Clock offset can be hours; DP handles missing events
Session > 30 min	Enable <code>timewarp=true</code>	Drift accumulates; linear warp prevents error growth

.18.2 Always validate sync quality

Before saving or using `sync_roi`, check:

```

1 % For analog sync: warpfactor should be very close to 1
2 assert(all(abs(sync_roi.warpfactor - 1) < 100e-6), ...
3         'warpfactor deviation > 100 ppm -- check sync channel');
4
5 % For event sync: mean_sim should be high
6 assert(all(sync_roi.mean_sim > 0.8), ...
7         'low similarity -- check event matching');
8
9 % After consolidation: residuals < 1 ms
10 cfg_check.timetol = 1e-3;
11 bml_sync_check(cfg_check, sync_roi_final);

```

.18.3 Annotation tables as the lingua franca

Every data structure in BML is an annotation table. This is intentional:

- Use `bml_annot_table()` to create any new table — it enforces `id`, `starts`, `ends`, `duration` and checks for overlaps.
- Never store times as arrays. Always put them in a table with `starts/ends` so they can be filtered, joined, and validated.
- `bml_annot_filter(A, B)` is the workhorse for restricting one table to the time range of another. Use it before any alignment to avoid processing irrelevant events.

.18.4 Save intermediate outputs

Sync is computationally expensive and hard to reproduce if session conditions change. Save:

```

1 save(fullfile(PATH_SYNC, 'sync_roi_final.mat'), 'sync_roi_final');
2 writetable(sync_roi_final, fullfile(PATH_SYNC, 'sync_roi_final.tsv'), ...
3           'FileType','text','Delimiter','\t');

```

Save the *un-consolidated* `sync_roi` from each method too (analog, neuroomega, ptb). If consolidation later fails for one file, you can re-run only that step.

.18.5 The fallback-first principle

When applying sync to event files, always prefer exact matches over interpolation (see Section .9.6). The priority order is:

1. **Exact Ripple timestamp** (matched via DP): hardware-precision ground truth.
2. **Linear warp** from the best DP-matched contiguous block: corrects offset + drift, accurate to $\sim 1\text{--}2$ ms.
3. **No sync** (NaN): flag events that fall outside the synced session window rather than extrapolating.

.18.6 BIDS event file conventions

All task event logs should follow BIDS:

- Columns: onset (seconds from task start), duration, trial_type, response_time (optional)
- After sync: save as `*-sync.tsv` with starts (master clock seconds) replacing onset
- `bml_annot_read.tsv` handles the onset $\rightarrow$ starts rename automatically

.18.7 Chunking strategy

- Chunk duration of 60–100 s is a good default. Shorter chunks have less data for correlation (noisier estimates); longer chunks miss local drift changes.
- For long sessions (>2 h), prefer 100 s chunks with `timewarp=true`.
- `chunk_extend` (extra seconds to load beyond the chunk boundary) should be at least as large as your expected OS timestamp error (typically 5–30 s).

.18.8 Toward Python: design principles to carry forward

The BML toolbox is currently implemented in MATLAB. The long-term goal is to migrate to Python (NumPy/Pandas/MNE). The following principles in the current design map naturally to Python:

MATLAB (BML)	Python equivalent	Key invariants
Annotation table	<code>pandas.DataFrame</code>	Always has starts, ends, duration columns
ROI table	<code>DataFrame + SyncCoords</code> dataclass	<code>s1, t1, s2, t2</code> define the linear map
<code>bml_idx2time</code>	Pure function $\rightarrow$ numpy vectorized	No side effects; invertible
<code>bml_annot_filter</code>	<code>df[df.starts >= t0 & ...]</code>	Interval overlap logic unchanged
<code>bml_annot Consolidate</code>	<code>groupby + merge</code>	Criterion function $\rightarrow$ lambda
DP matching	<code>scipy</code> or pure Python	Identical recurrence; only similarity function changes
<code>FT_DATATYPE_RAW</code>	<code>mne.io.RawArray</code>	Single trial, single channel for sync functions

When porting, preserve the two-coordinate system (`s1, t1, s2, t2`): it is the core abstraction that decouples sample indices from master-clock times, and nothing in the Python ecosystem provides an equivalent out of the box.

.1 Mathematical Reference

Index-to-time and inverse

$$F_{s,\text{eff}} = \frac{s_2 - s_1}{t_2 - t_1}, \quad t(i) = \frac{i}{F_{s,\text{eff}}} - \frac{0.5}{F_{s,\text{eff}}} + \frac{s_2 t_1 - t_2 s_1}{s_2 - s_1}$$

$$i(t) = \text{round}\left(\frac{t_2 s_1 - s_2 t_1 + (s_2 - s_1)t}{t_2 - t_1}\right)$$

Time-warp model and cost

$$w(t) = t_{\text{pivot}} + wt_0 + (t - t_{\text{pivot}}) \cdot ws_1$$

$$\mathcal{L}(wt_0, ws_1) = -\frac{\langle f(w(\mathbf{t})), p \rangle}{\|f(w(\mathbf{t}_0))\| \|p\|} + \left(\frac{wt_0}{P_{\delta t}}\right)^2 + \left(\frac{ws_1 - 1}{P_{\xi}}\right)^4$$

$$t_1^{\text{new}} = t_{\text{pivot}} - wt_0 - \frac{t_2^{\text{crop}} - t_1^{\text{crop}}}{2ws_1}, \quad t_2^{\text{new}} = t_{\text{pivot}} - wt_0 + \frac{t_2^{\text{crop}} - t_1^{\text{crop}}}{2ws_1}$$

Event alignment cost (`bml_timealign_annot`)

$$\mathcal{C}(\delta t) = \sqrt{\sum_i \left[\min\left(\min_j |s_i + \delta t - m_j|, t_{\text{clip}}\right) \right]^2}$$

Event similarity (new 5-feature, `bml_sync_match_events`)

$$\text{sim}(\mathbf{x}_i, \mathbf{x}_j) = \frac{1}{W} \left[w_1 \cdot \frac{1}{1 + (\Delta t^{\text{pre}}/\tau)^2} + w_2 \cdot \frac{1}{1 + (\Delta t^{\text{post}}/\tau)^2} + w_3 \cdot \mathbb{I}[v_1^{\text{pre}} = v_2^{\text{pre}}] + w_4 \cdot \mathbb{I}[v_1^{\text{post}} = v_2^{\text{post}}] + w_5 \cdot \mathbb{I}[v_1^{\text{pre}} = v_2^{\text{post}}] \right]$$

DP recurrence

$$\text{dp}[i+1, j+1] = \max(\text{dp}[i, j] + \text{sim}(\mathbf{x}_i, \mathbf{x}_j), \text{dp}[i+1, j], \text{dp}[i, j+1])$$